ExactKV 的 LMCache 实现：codec 怎么进 cache server，FragPack 怎么进 vLLM 的 GPU 池

2026-11-05 | Fragpack · feat/lmcache-056-lossless-kv · c0b2273

前置阅读：《ExactKV 论文稿：无损 KV 压缩进缓存层和 attention》《vLLM 的 KV 管理：spec、分组、布局、块池和 TP / PP / DCP / PCP 切分》

目录

1. 两种模式、进程和配置
   1.1 tier mode 压 GPU 以外的层，pool mode 连 GPU 池一起压
   1.2 进程、线程和 stream
   1.3 配置项和启动时的分流
2. tier mode 的 record 格式
   2.1 一个 chunk 切成 96 个平面、每个平面 128 个 tile
   2.2 码本：每个（层，K/V，KV 头）15 项，7 个主码、7 个次码、1 个哨兵
   2.3 record 的字节布局：8 KiB 头加 96 段平面数据
   2.4 长度怎么算，什么时候存 raw
   2.5 FP8 record：符号 + 指数按通道移位后查码本，尾数单独成一条 3 bit 流
3. GPU 上的编码和解码
   3.1 编码：count、layout、compact 三次 launch，长度最后读一次
   3.2 解码：一个 warp 一个 tile，PRMT 查表，128 bit 直接写进块
   3.3 KVRestorePlan：解码的元数据只上传一次
   3.4 直接读写引擎块：vLLM 0.30 的层张量首尾相接
4. tier mode 的 store：handler 只 launch，长度由 store worker 读
   4.1 原来的 store：注册时定好对象大小，提交挂在 stream 回调上
   4.2 codec store 的调用链：两条线程、两条 stream
   4.3 store pool：默认 128 个 chunk，够两个 step
   4.4 StoreTally：一次 store 只上报一次
   4.5 L1 和 L2 按 record 长度存
5. tier mode 的 retrieve：长度来自主机，拷贝跑在解码前面
   5.1 retrieve 的调用链
   5.2 四套 record buffer：拷贝不排在解码后面
   5.3 什么时候直接解进块，什么时候走 scratch
6. pool mode：FragPack attention backend

6.1 怎么接进 vLLM：插件入口、KV spec 和一组限制
6.2 record 和 page：按平均长度定页大小，CAS 抢空间
6.3 每一步：prepare、逐层 write / attend / encode、commit
6.4 OpenBlocks：从调度器输出镜像每个请求的块表
6.5 decode attention：所有请求共用一个段长
6.6 prefill：解进 BF16 workspace 再跑 FA3
7. pool mode 接 LMCache：块当 blob 存
7.1 握手：layout hint 先到，store 的表后到
7.2 blob 格式：4 KiB 头，后面接 192 条 record
7.3 store 和 retrieve：export 进 pool，import 失败就重算

1. 两种模式、进程和配置

论文的方法（《ExactKV 论文稿：无损 KV 压缩进缓存层和 attention》）落在一个基于 LMCache 0.5.6 的分支上：14 个提交，加了约 2.03 万行：codec 包 `lmcache/v1/lossless_kv/` 约 1.08 万行，LMCache 其余部分（cache server、存储层、vLLM 接入）约 1.8 千行，测试 3.1 千行，评测 harness 4.6 千行。论文的码一句话回顾：BF16 的低字节原样存；高字节每个（层，K/V，KV 头）一张 15 项码本，最常见的 7 个值用 3 bit 主码，接下来 7 个再用 3 bit 次码，剩下的存原始字节。代码用这套码做了两种部署，可以单独开，也可以一起开。[读码]

1.1 tier mode 压 GPU 以外的层，pool mode 连 GPU 池一起压

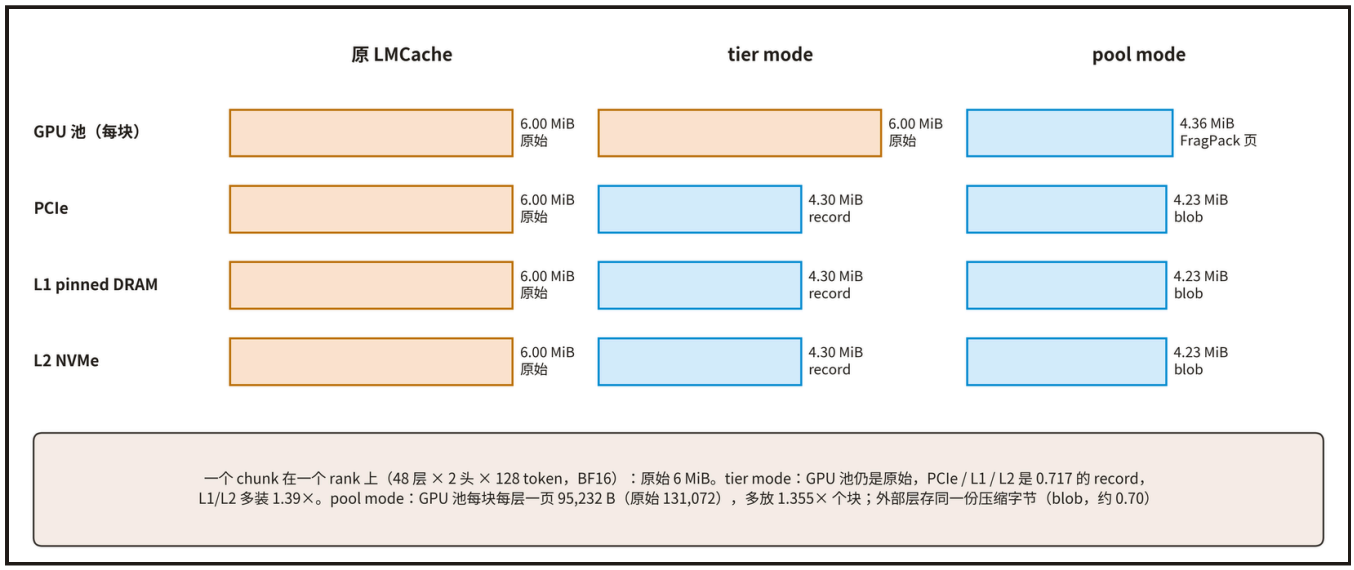


图 1-1 一个 chunk 在三种部署下每一段存多少字节

tier mode：vLLM 不改。LMCache 的 cache server 从 vLLM 的显存里拿 KV 时先编码，送回去时再解码，所以 PCIe、L1（pinned DRAM）和 L2（NVMe）上走的都是压缩后的 record。开关是 cache server 的 `--lossless-kv-codec`。vLLM 自己的 GPU 池仍是原始 KV。
读码 `lmcache/v1/multiprocess/lossless_kv_transfer.py`

pool mode：vLLM 的 GPU 池本身存压缩数据。LMCache 包里带了一个 attention backend（`FragPackAttentionBackend`），经 vLLM 的插件入口注册成 `--attention-backend CUSTOM`；decode attention 在 kernel 里边解边算。GPU 池里已经是压缩的块，cache server 不再跑 codec，把每个块的压缩字节原样当一个 blob 存进 L1/L2。这条路径由 vLLM 一侧的 backend 自动打开，cache server 不需要开 `--lossless-kv-codec`。
读码 `lmcache/v1/lossless_kv/fragpack/backend.py`
`lmcache/v1/multiprocess/fragpack_blob_transfer.py`

两种模式的码相同（3 bit 主码加转义），也共用一次离线标定：每个（层，K/V，KV 头）一张按频率排好的 22 项表，FragPack 用全部 22 项，tier 取前 14 项（2.2 节）。标量的排列不同：tier record 按平面连续排，方

便整块解进分页 KV；FragPack record 按 tensor core 的 fragment 顺序排，方便 attention kernel 每个线程直接读自己的字节（6.2 节）。

本文的例子贯穿全文：

项	取值
模型	Qwen3-30B-A3B-Instruct-2507：48 层，32 个 query 头、4 个 KV 头，head_dim 128，BF16 KV
并行	TP2，两张 H800（132 个 SM），每个 rank 2 个 KV 头
块	vLLM `--block-size 128`，LMCache `--chunk-size 128`，一个 chunk 正好一个块
容量	每卡 KV 预算 54 GiB，L1 400 GiB，L2 2 TiB（harness 的默认值）

一个 chunk 在一个 rank 上的原始大小：48 层 × 2（K、V）× 128 token × 2 头 × 128 维 × 2 字节 = 6,291,456 B = 6 MiB。两个 rank 各存各的，所以一个 128 token 的 chunk 在 L1 里是两个对象。按论文的标定码本（主码转义率 6.1%、raw 率 0.19%）算，tier record 是 4,513,792 B（原始的 0.717，2.4 节）；pool mode 的 blob 平均约 0.70（7.2 节）。

1.2 进程、线程和 stream

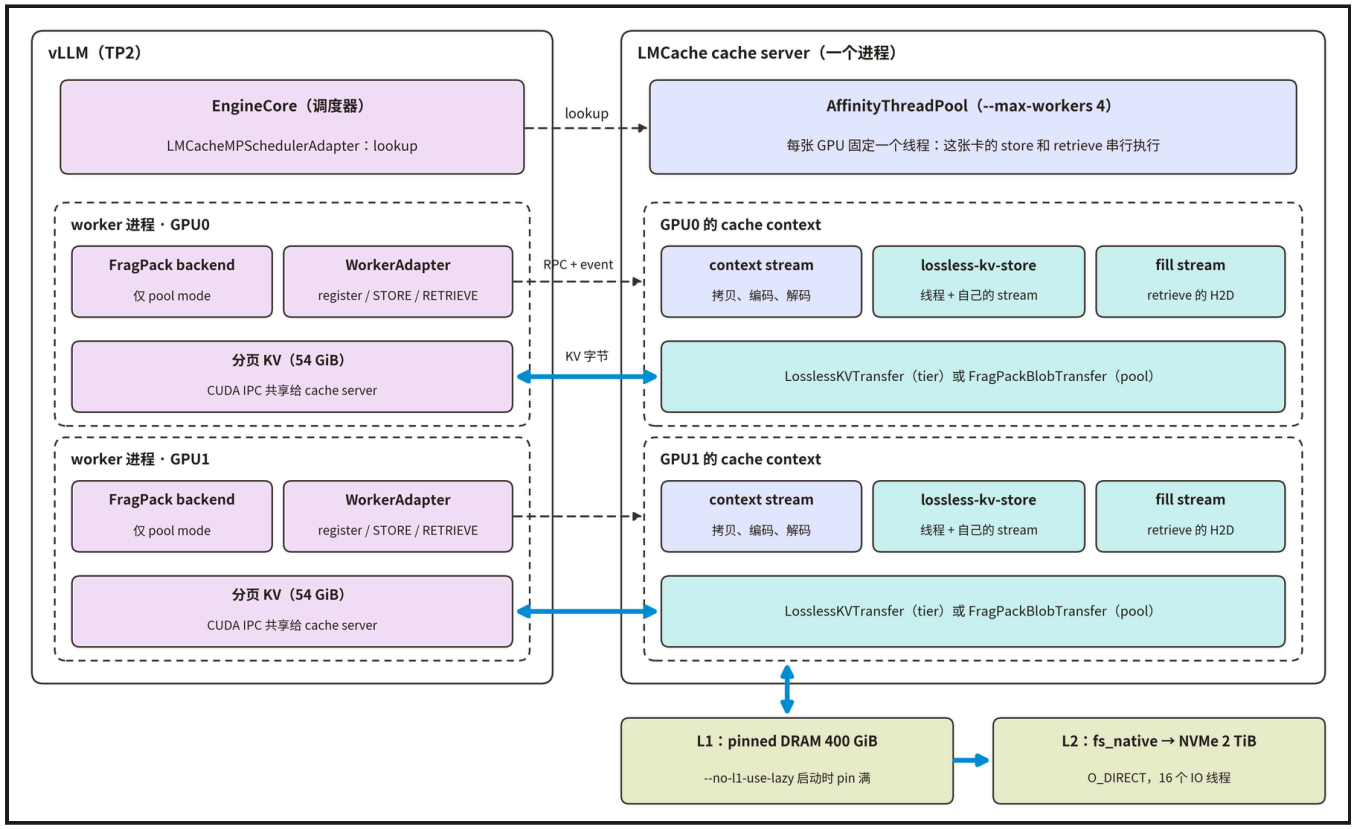


图 1-2 vLLM、LMCache cache server 和存储层的进程与线程

LMCache 的多进程模式（MP）里，cache server 是一个独立进程，和 vLLM 跑在同一台机器上。[2]

- **vLLM 的 EngineCore**：调度器里的 `LMCacheMPSchedulerAdapter` 对每个新请求向 cache server 发 lookup，得到命中的 token 数；命中但只在 L2 的 chunk 由 server 提前预取进 L1。
  读码 `lmcache/integration/vllm/vllm_multi_process_adapter.py`
- **vLLM 的 worker 进程（每卡一个）**：`LMCacheMPWorkerAdapter` 启动时把自己的分页 KV 用 CUDA IPC 句柄注册给 server（`register_kv_cache`），之后每步在 forward 前发 retrieve、forward 后发 store。两个 RPC 都带一个 CUDA event 的 IPC 句柄，server 在自己的 stream 上等这个 event，所以拷贝排在产生这批 KV 的 forward 之后。
- **cache server**：每个注册过的 GPU 有一个 cache context，自带一条 CUDA stream（下称 context stream）。store 和 retrieve 是 `HandlerType.BLOCKING` 且 `requires_client_affinity=True` 的 handler，由 `AffinityThreadPool` 按 GPU 固定分给一个线程（harness 开 `--max-workers 4`，TP2 用其中两个）。所以同一张卡的 store 和 retrieve 在同一个线程上排队执行，这是 4.2 节整个设计的出发点。
- **codec 新增的线程和 stream**：每个 cache context 再有一个 `lossless-kv-store` 线程和它自己的 stream（4.2 节），retrieve 另有一条 fill stream（5.2 节）。
- **存储层**：L1 是 server 进程里的 pinned DRAM 池（`--no-l1-use-lazy` 让它启动时一次 pin 满）；L2 是 `fs_native` adapter，用 O_DIRECT 写文件，16 个 IO 线程。

代码改动的位置：

```
LMCache 0.5.6 + lossless-kv (91ec45c)
├── lmcache/v1/lossless_kv/                    codec 包（两种模式共用码）
│   ├── format.py           record 头、平面描述符、码表格式、对齐、长度公式
│   ├── codec.py            CPU 参考编解码、通用码本
│   ├── calibration.py      两种模式共用的离线标定、artifact 读写、按 TP 分片取 tier 码表
│   ├── tilelang_kernels.py   GPU 编码三步、解码一步（TileLang）
│   ├── compressed_kv.py    FusedOnlineKVPacker、FusedCompressedKVDecoder、KVRestore
│   ├── chunk_codec.py      KVMajorChunkCodec：按 LMCache chunk 编解码
│   ├── kv_layout.py        vLLM 各种 KV 布局的逻辑 / 物理 view
│   └── fragpack/           pool mode
│       ├── backend.py      vLLM attention backend、OpenBlocks
│       ├── hbm_kv.py / .cu  GPU 池里的 record 存储、blob 导出导入
│       ├── attn_page.py    FragPack record 和 blob 格式
│       ├── attn_tables.py 每（层, K/V, 头）的标定算法、FragPack 的设备端码表
│       ├── fused_attn_decode.py / .cu  融合 decode attention
│       ├── registry.py     backend 和 connector 之间的交接
│       └── plugin.py       vllm.general_plugins 入口
├── lmcache/v1/multiprocess/
│   ├── config.py           LosslessKVConfig 和五个命令行开关
│   ├── lossless_kv_transfer.py   tier mode 的 store / retrieve、RecordStoreWorker
│   ├── fragpack_blob_transfer.py pool mode 的 blob store / retrieve
│   └── modules/lmcache_driven_transfer.py  在原 store / retrieve 里分流
├── lmcache/v1/distributed/   按对象实际大小预留（约 160 行）
├── lmcache/integration/vllm/ layout hint、register_fragpack_store
├── tests/v1/lossless_kv/、tests/v1/multiprocess/   单元和 GPU 往返测试
└── benchmarks/lossless_kv/   评测 harness：arms.py、services.py、run_arm.sh …
```

1.3 配置项和启动时的分流

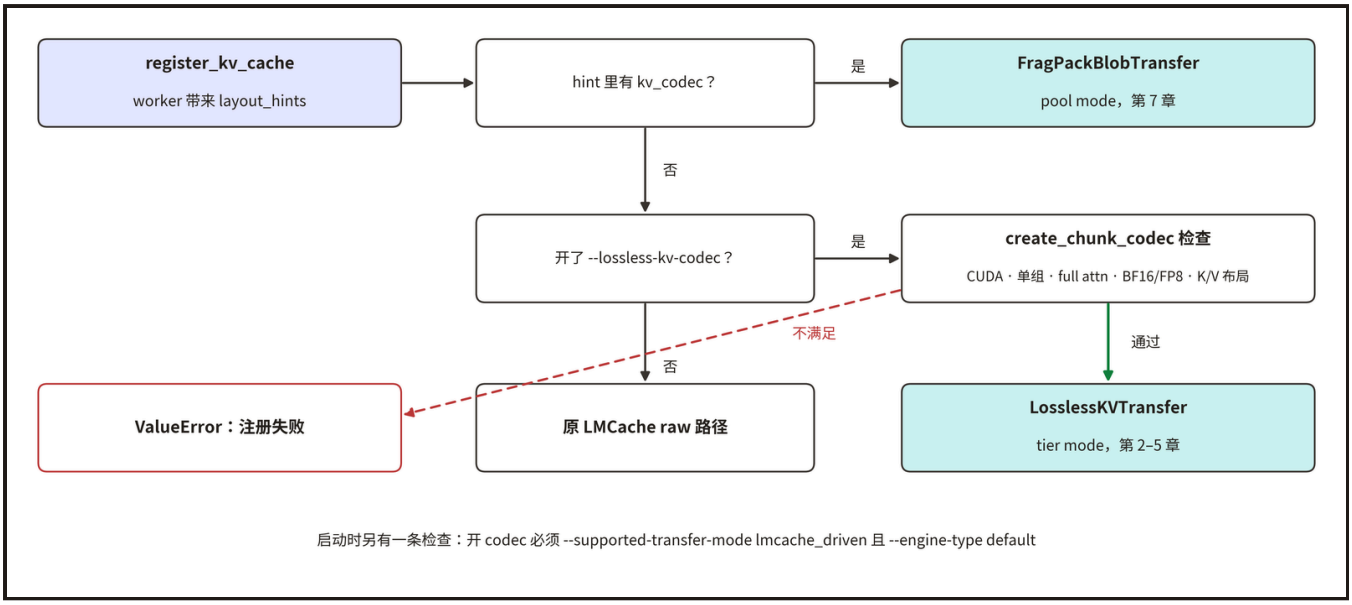


图 1-3 注册 KV cache 时选哪种 transfer

tier mode 的五个开关都在 `LosslessKVConfig` 里，`MPServerConfig` 带一份：

读码 `lmcache/v1/multiprocess/config.py`

开关	默认	作用和取值的理由
`--lossless-kv-codec`	关	每个 chunk 存成压缩 record
`--lossless-kv-codebooks DIR`	无（用通用码本）	这个模型离线标定的 artifact 目录（`tables.npz` + `metadata.json`，pool mode 读同一个 `tables.npz`，2.2 节）；论文测得这个模型用通用码本是 .771、标定后 .716[1]
`--lossless-kv-batch N`	8	一次编解码多少个 chunk。编码每批同步主机一次读长度，批越宽同步越少；8 个 chunk 在例子里是 48 MiB，一个 vLLM 默认 8192 token 的 step 正好 8 批（4.3 节）
`--lossless-kv-store-pool N`	128	store worker 的缓冲按 chunk 计，至少一批；128 是两个 8192 token 的 step（4.3 节）[3]
`--lossless-kv-capacity R`	无（变长）	把每个 record 补齐到 align4K(raw × R)，放不下就整个 store 失败（2.4 节）。给只能分配一种对象大小的引擎用，论文的实验不用

`MPServerConfig.__post_init__` 在启动时检查：开 codec 必须是 `--supported-transfer-mode lmcache_driven` 和 `--engine-type default`，否则 `ValueError`。原因是 codec 挂在「server 自己从 vLLM

显存拷 KV」这条路径上；engine-driven 模式下拷贝由 worker 做，server 拿到的已经是主机上的字节，没有 GPU 可以编码。［读码］

注册时分流：`register_kv_cache` 调 `_create_record_transfer`，先看 worker 发来的 layout hint 里有没有 `kv_codec`。

1. 有 `kv_codec`（取值 `fragpack` 或 `fragpack-fp8`，由 pool mode 的 backend 在 vLLM 里写进 hint，7.1 节）：建 `FragPackBlobTransfer`（fragpack_blob_transfer.py，pool mode 下按 blob 存取块，第 7 章），不管 codec 开没开。GPU 池里已经是压缩块，不能按原始布局拷。
2. 否则开了 codec：建 `LosslessKVTransfer`（lossless_kv_transfer.py，一个 cache context 一个，负责 tier mode 的 store 和 retrieve，第 4、5 章）。它的构造函数调 `create_chunk_codec`（输入 cache context 和配置，输出一个 `KVMajorChunkCodec`，2.1 节），不满足就抛 `ValueError`、注册失败：CUDA 设备；只有一个 object group 和一个 kernel group（不支持 SWA、Mamba 这类混合模型）；full attention；dtype 是 BF16、`float8_e4m3fn` 或 `uint8`（vLLM 的 FP8 cache 按字节存）；K/V 分开（`kv_size == 2`）或交错（`kv_size == 1`，每个头里 K 后面跟 V）。最后核对 codec 算出的原始 chunk 字节数和 LMCache 自己暂存一个 chunk 的字节数相等，例子里两边都是 6,291,456。
   `LosslessKVTransfer` 还从 layout hint 里取 `kv_head_start`（这个 rank 的第一个 KV 头），用来从标定 artifact 里挑这个 rank 的码表行（2.2 节）。
3. 都没有：原样走 LMCache 的 raw 路径。

注册成功后，transfer 对象按 `id(cache_context)` 存进 `_lossless_transfers`，store 和 retrieve 的 handler 每次先查它，有就分流到 codec 或 blob 路径，没有就走原代码。反注册（或 worker 失联被回收）时先 `close()` transfer，再关 cache context：store worker 里排着的批持有引擎块的拷贝，要先存完。
读码 `lmcache/v1/multiprocess/modules/lmcache_driven_transfer.py`

2. tier mode 的 record 格式

tier mode 把一个 chunk 在一个 rank 上的 KV 编成一个 record，做这件事的是 `KVMajorChunkCodec`（chunk_codec.py，cache server 里每个 cache context 一个：输入 LMCache 的一批 chunk，输出每个 chunk 的 record 和长度；反过来把 record 解回 chunk 或直接解进 vLLM 的块）。例子里 6 MiB 的原始 chunk 用标定码本编成约 4.30 MiB。record 是 LMCache 里一个对象的全部内容，L1 和 L2 都原样存它，所以格式同时要满足三件事：GPU 上能从任意 tile 开始并行解码，O_DIRECT 能直接读写，最坏情况不比原始大。
读码 `lmcache/v1/lossless_kv/format.py`

2.1 一个 chunk 切成 96 个平面、每个平面 128 个 tile

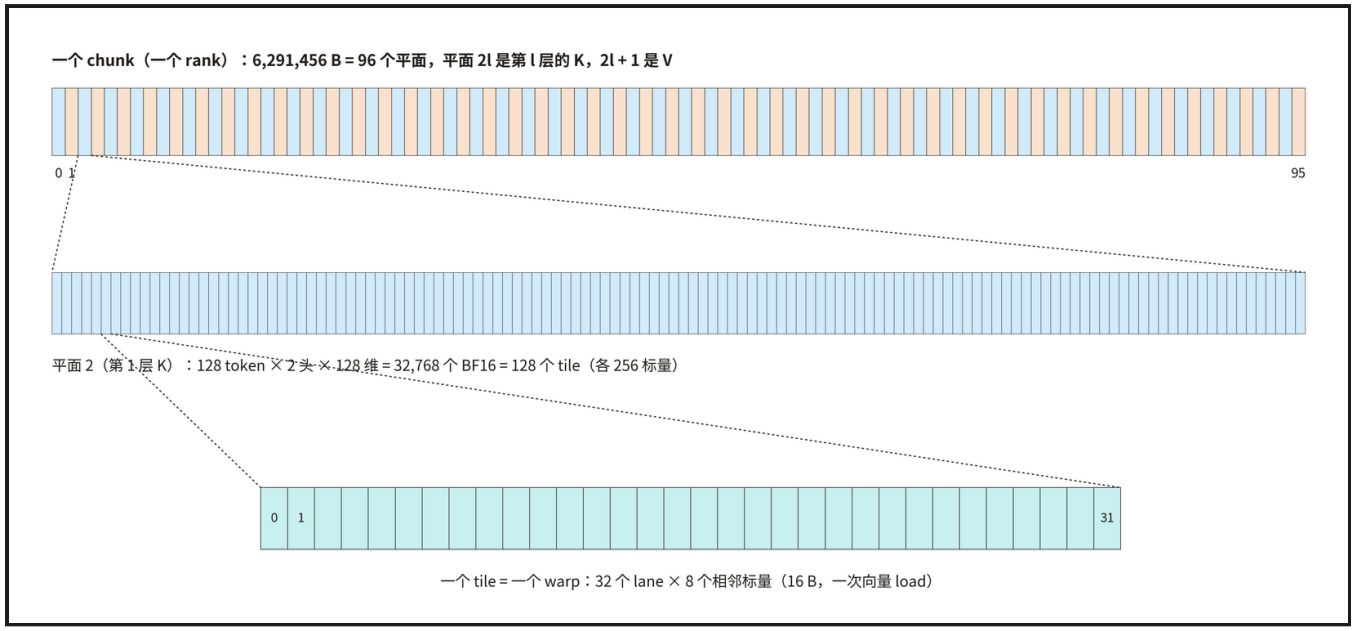


图 2-1 一个 chunk 怎么切成平面、tile 和 lane

解决什么：一个 chunk 有 314 万个标量，解码要分给几千个线程同时做，每个线程得知道自己的输入从哪开始；码本也要贴合数据的分布，而不同层、K 和 V、不同 KV 头的分布都不同。

切法：`CompressedStoreGeometry`（format.py，一个 chunk 的几何：层数、头数、head_dim、标量字节数、tile 大小）把 chunk 按（层，K 或 V）切成平面。

- 平面数 = 层数 × 2 = 96。每个平面是一层的 K（或 V）在 128 token × 2 头 × 128 维上的全部标量，32,768 个。平面里每个 KV 头各用一张码本，例子一个平面 2 张、一个 rank 共 192 张（2.2 节）。
- tile 是 256 个标量（`ONLINE_TILE_SCALARS`），一个平面 128 个 tile。tile 是能独立解码的最小单位：一个 warp 解一个 tile，32 个 lane 每个负责 8 个相邻标量（8 个 BF16 正好 16 字节，一次向量 load）。所以 256 = 32 × 8，tile 再大一个 warp 就要循环，再小前缀表就变大（每个 tile 8 字节，2.3 节）[推断]。
- 原始 chunk 必须 4 KiB 对齐（`slot_size % 4096`），不然构造时报错：record 要能 O_DIRECT 读写，原始大小又是 raw 兜底时的长度。例子里 6,291,456 = 1536 × 4096。

布局：vLLM 0.30 的 FlashAttention cache 每层一个 `[块, 128 token, 头, 2 × 128]` 的张量，每个头里 K 的 128 维后面紧跟 V 的 128 维（`kv_size == 1`，LMCache 叫 fused K/V）。record 按平面排，所以编码时 K 和 V 要按步长分开读：kernel 拿的是一个逻辑 view `[块, 层, 2, token, 头, 维]`，物理顺序是 `[层, 块, token, 头, 2, 维]`。`kv_layout.py` 里的 `layer_major_kv_view` 负责把 vLLM 背靠背分配的 48 个层张量看成这样一个 view，前提是它们在同一块分配里首尾相接（3.4 节）。

读码 `lmcache/v1/lossless_kv/kv_layout.py`

2.2 码本：每个（层，K/V，KV 头）15 项，7 个主码、7 个次码、1 个哨兵

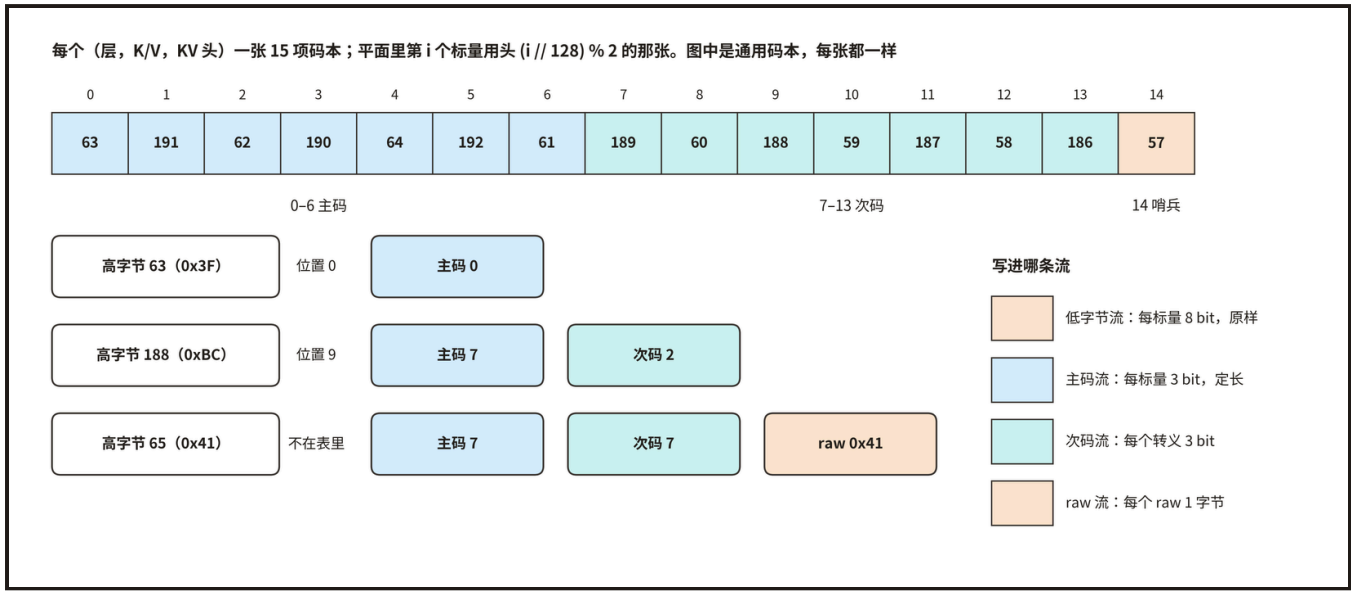


图 2-2 每头一张的码本和一个高字节的三条去路

编码什么：BF16 只编高字节（符号 + 指数高 7 位），低字节（指数最低位 + 7 位尾数）原样存。码本按（层，K/V，KV 头）各一张，每张 15 个字节值（`CODEBOOK_ENTRIES = 15`）。平面里第 i 个标量属于第 `(i // head_dim) % heads` 个头，用那个头的表：

码本位置	用法	一个高字节落在这里时写什么
0–6	主码	主码流写 3 bit 的 0–6
7–13	次码	主码写 7（转义），次码流写 3 bit 的 0–6
14 和表外的值	raw	主码写 7，次码写 7，raw 流写原始高字节

编码器在启动时把每张码本展开成一张 256 项的查找表（`_lookup`，例子一个 rank 192 × 256 = 48 KiB），表外的值和第 15 项走同一条 raw 路。第 15 项只是占位，保证每行 15 个值互不相同，它本身也编成 raw。

码表格式：codec 拿到的是一段字节（`format.codec_tables_bytes`）：先是 `行数 × 15` 的码本，再是 `行数 × head_dim` 的 FP8 指数偏移（2.5 节，BF16 全是 0）。行号 `(2 × 层 + K/V) × 头数 + 头`。例子一个 rank 192 行，192 × (15 + 128) = 27,456 B。`split_codec_tables` 拆开时检查长度、每行 15 个值不重复、FP8 的码本值小于 32 且偏移小于 16、BF16 的偏移全是 0，不满足就 `ValueError`，KV cache 注册失败。

标定：两种模式共用一次离线标定（`benchmarks/lossless_kv/calibrate.py` 调 `calibrate_codebooks`，标定算法在 FragPack 的 `attn_tables.calibrate` 里）。

1. 输入是 `capture_kv.py` 从真实请求抓下来的 BF16 位模式 `[块, 层, 2, 128 token, 全部 KV 头, head_dim]`，必须包含模型的全部 KV 头，不是某个 rank 的。

2. 对每个（层，K/V，头）统计符号直方图，按频率从高到低取 22 个（`TABLE_ENTRIES`，频率相同按值小的在前），写进 `table[L, 2, H, 22]`。FP8 还同时拟合 `offset[L, 2, H, head_dim]`（2.5 节）。
3. 结果存成 `tables.npz`（`table`、`offset`、`fp8`）加 `metadata.json`（artifact 版本 2、层数、头数、head_dim、标定用了多少块、RoPE 参数、对三个数组算的 SHA-256）。

FragPack 用每张表的全部 22 项（7 个主码 + 15 个 nibble 扩展值，6.2 节），tier 取前 14 项（7 个主码 + 7 个次码）再补一个哨兵：`tier_codebooks` 对每行取前 14 个值（FP8 先右移 3 位成 5 bit 符号），哨兵取不在这 14 个里的最小值。前 14 项正好是 FragPack 那张表的前缀，所以一份 artifact 两边都能用：tier 的 `--lossless-kv-codebooks` 给目录，pool 的 `LMCACHE_FRAGPACK_TABLES` 给目录里的 `tables.npz`。以前两边各标定一次，tier 的码本按平面在所有头上汇总；共用之后两边的码表粒度一样。[读码] [8]

每头一张值不值：在这个模型的 agent KV 上，BF16 从按平面一张换成每头一张，比率 .7163 → .7161，几乎不变。BF16 的变长部分在标定码本下只有约 0.2 bit/标量（2.4 节），能省的本来就不多。好处主要在 FP8：和逐通道偏移一起，比率 .900 → .868（2.5 节）。代价是表变大（一个 rank 15 × 96 = 1,440 B 变成 27,456 B，加上 shared memory 里每个头一张 256 项的表，3.1 节），都是启动时的一次性开销。[实测] `codec_pareto_bench`，Qwen3-30B-A3B agent KV 抓取

TP 分片：artifact 存的是模型全部 KV 头，每个 rank 只取自己的那几行。vLLM 给 TP 的第 r 个 rank（共 t 个）分模型 H 个头里从 `r × H // t` 开始的连续一段，H < t 时几个 rank 复制同一个头。worker 的 `vllm_layout_hints` 把这个起点作为 `kv_head_start` 放进 layout hint（rank 0 是 0，不发），cache server 建 `LosslessKVTransfer` 时传给 `load_calibrated_codebooks(dir, geometry, head_start)`，它切出 `[head_start, head_start + 本 rank 头数)` 这几行。例子 H = 4、t = 2：rank 0 取头 0–1，rank 1 取头 2–3。

以前按平面汇总的码本和 TP 无关，任何 TP 都能用；现在必须知道 rank 的头号，起点算错会用别的头的表编码。结果仍然无损（表外的值走 raw），只是比率变差。

加载时检查：`load_artifact_tables` 检查 artifact 版本是 2、SHA-256 对得上；`load_calibrated_codebooks` 再检查层数、head_dim、dtype（FP8 还是 BF16）和这个 rank 的头号范围。任何一项不对都 `ValueError`，注册失败。旧格式的 artifact（`codebooks.bin`）版本不对，直接被拒，要重新标定。

通用码本：不给 `--lossless-kv-codebooks` 时每一行都是同一张 `_GENERIC_ONLINE_CODEBOOK`：63, 191, 62, 190, 64, 192, 61, 189, 60, 188, 59, 187, 58, 186, 57，偏移全 0，码表长度和标定的一样。63（0x3F）是正数、绝对值在 [0.5, 2) 的高字节，191（0xBF）是它的负数，之后正负交替往两边扩：62 是 [0.125, 0.5)，64 是 [2, 8)。在线服务没法先跑一遍标定，所以默认表是固定的；它只影响比率，不影响正确性，表外的值都走 raw。 **读码** `lmcache/v1/lossless_kv/codec.py`

为什么分两级：一级 3 bit 只能表示 7 个值，覆盖约 94% 的标量；剩下 6% 再给一级 3 bit，14 个值以外只剩约 0.2%。这样主码流长度固定（每标量 3 bit），GPU 上每个 lane 不用知道前面的变长码就能定位自己的主码；只有转义的标量才需要前缀和去找次码和 raw 的位置（3.2 节）。论文第 3.3 节比较过这个选择和变长码。

2.3 record 的字节布局：8 KiB 头加 96 段平面数据

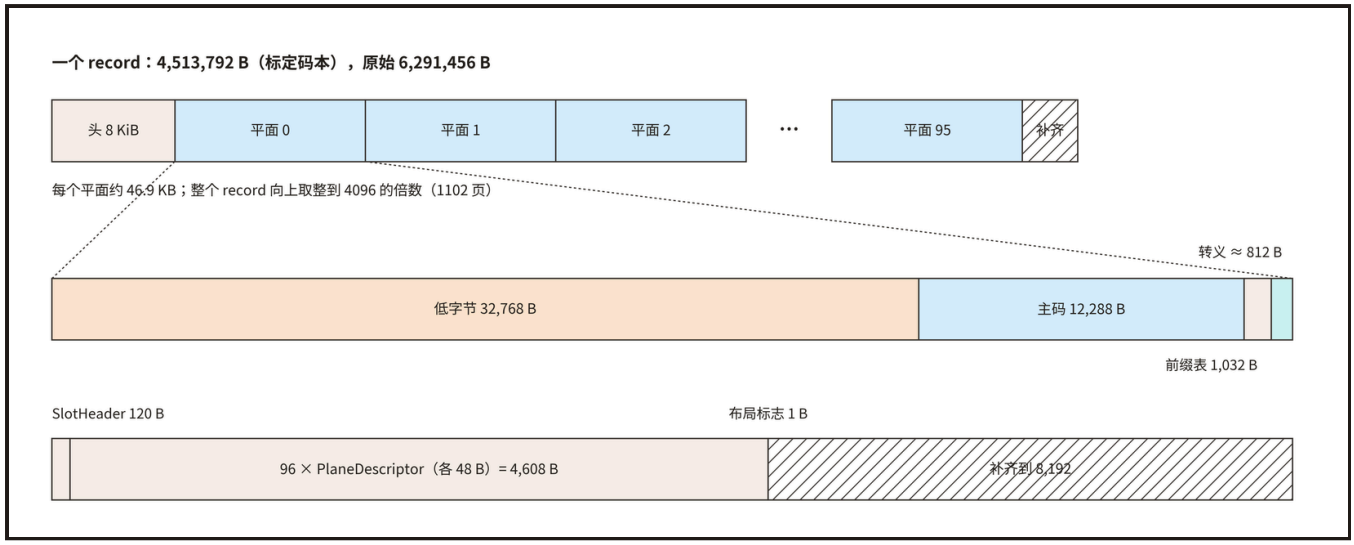


图 2-3 例子里一个 record 的字节布局

段	大小（例子）	内容
头	8,192 B	`SlotHeader`（120 B：magic `DKVSLOT1`、格式版本 2、头长、对齐 4096、tile 大小、slot id、层数、平面数、原始长度、存储长度、码表 SHA-256（码本和 FP8 偏移一起算）、原始数据哈希）+ 96 个 `PlaneDescriptor`（各 48 B：层、K/V、标量数、tile 数、本平面在 record 里的偏移和长度、各流偏移、转义数）+ 1 字节布局标志
每个平面：低字节流	32,768 B	每标量 1 字节，原样
每个平面：主码流	12,288 B	每标量 3 bit
每个平面：前缀表	1,032 B	两张 `uint32` 表，各 129 项：每个 tile 之前累计多少个转义、多少个 raw
每个平面：转义流	变长	次码（每个转义 3 bit），后接 raw 高字节（每个 1 字节）
补齐	到 4 KiB	整个 record 向上取整到 4096 的倍数

头为什么是两页：120 + 96 × 48 + 1 = 4,729 B，超过一页，`slot_header_bytes` 向上取整到 8,192。41 层（82 个平面）以内是一页（120 + 82 × 48 + 1 = 4,057）。头长写在头里，所以深模型和浅模型的 record 用同一个解析器。

布局标志：头后面那一个字节记 `SYMBOL_BITS | 0x80 | 0x40`（3 bit 主码、转义流打包、3 bit 次码），FP8 再或上 0x20。解码前核对，一个 FP8 record 不会被当成 BF16 解。

谁写头：编码最后一步 compact kernel 的平面 0 CTA 直接在 GPU 上序列化头（3.1 节），热路径上没有主机拷元数据。

2.4 长度怎么算，什么时候存 raw

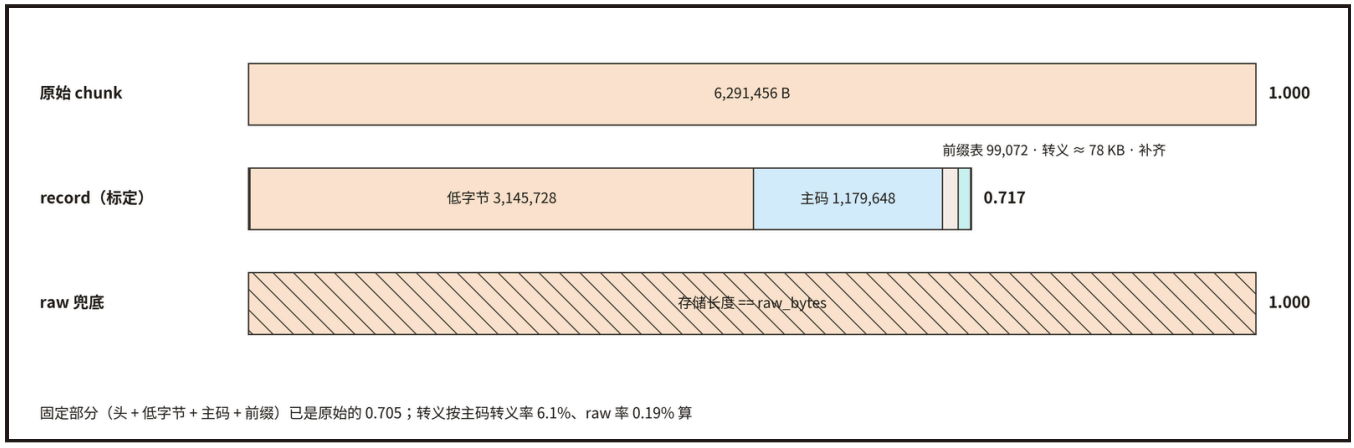


图 2-4 例子里 record 的长度组成

公式：`FusedOnlineKVPacker._plan_spans` 在主机上按设备回传的计数算每个 record 的长度：

长度 = align4K(头 + Σ平面 (低字节 + 主码 + 前缀表 + ⌈转义数 × 3 / 8⌉ + raw 数))

固定部分每平面 32,768 + 12,288 + 1,032 = 46,088 B，96 个平面 4,424,448 B，加头 4,432,640 B，已经是原始的 0.705。变长部分按论文这个模型的标定码本（主码转义率 6.1%、raw 率 0.19%）[1]：每平面 ⌈32,768 × 0.061 × 3 / 8⌉ + 32,768 × 0.0019 ≈ 750 + 62 = 812 B，96 个平面约 78 KB。合计 4,510,617 B，取整到 4,513,792 B（1102 页），是原始 6,291,456 B 的 0.717。

所以 tier record 的比率几乎由固定部分决定：低字节占 8 bit、主码 3 bit、前缀 0.25 bit，变长部分在标定码本下只有约 0.2 bit。

raw 兜底：判断在设备上做，按整个 chunk 判，不按平面判。layout kernel 的一个 warp 把一个 chunk 的 96 个平面长度加起来，取整到 4 KiB；结果不小于原始大小，就把这个 chunk 标成 overflow，compact kernel 把 96 个平面原样背靠背拷进 record。按 chunk 判是因为各平面的转义密度不同（FP8 的平面之间差 10% 以上），给每个平面一个均分的预算会让个别密的平面把整个 chunk 拖成 raw，而总长其实放得下。启动时 `online_fixed_envelope_geometry` 只检查一件事：一个没有任何转义的 record 放得进原始大小，放不进说明这个几何压不了，构造时直接报错。 **读码** `lmcache/v1/lossless_kv/compressed_kv.py`

条件是「取整后不小于」而不是「大于」：编码成功但取整后没变小的 chunk 存成压缩版不省空间，retrieve 时还要多跑一次解码，不如存 raw。这一条是 37c499d 改的，之前是 `record_cursor > slot_stride`。[5]

raw chunk 的存储长度就是原始大小，没有单独的标志位：`KVMajorChunkCodec` 的约定是「长度 == `raw_bytes` 就是 raw」。retrieve 看到这个长度就跳过解码（5.3 节）。从引擎块直接打包时，raw 的 record

是 96 个平面背靠背，和 LMCache 原本暂存的 fused chunk 顺序不同，`engine_records` 会在 stream 上把它转回 chunk 布局，所以 L1 里的 raw chunk 和 LMCache 原来存的字节一样。

读码 `lmcache/v1/lossless_kv/chunk_codec.py`

固定容量模式：`--lossless-kv-capacity R` 时每个 record 补齐到 `min(raw, align4K([raw × R]))`，所有对象一样大；放不下（或要走 raw）就抛 `ChunkCapacityError`，整个 store 失败、什么都不存。它不能存 raw 兜底，因为容量类比原始小。这个模式给只能分配一种对象大小的引擎留的，从引擎块直接打包也不支持它（打包要原始大小的缓冲原地补齐）。

2.5 FP8 record：符号 + 指数按通道移位后查码本，尾数单独成一条 3 bit 流

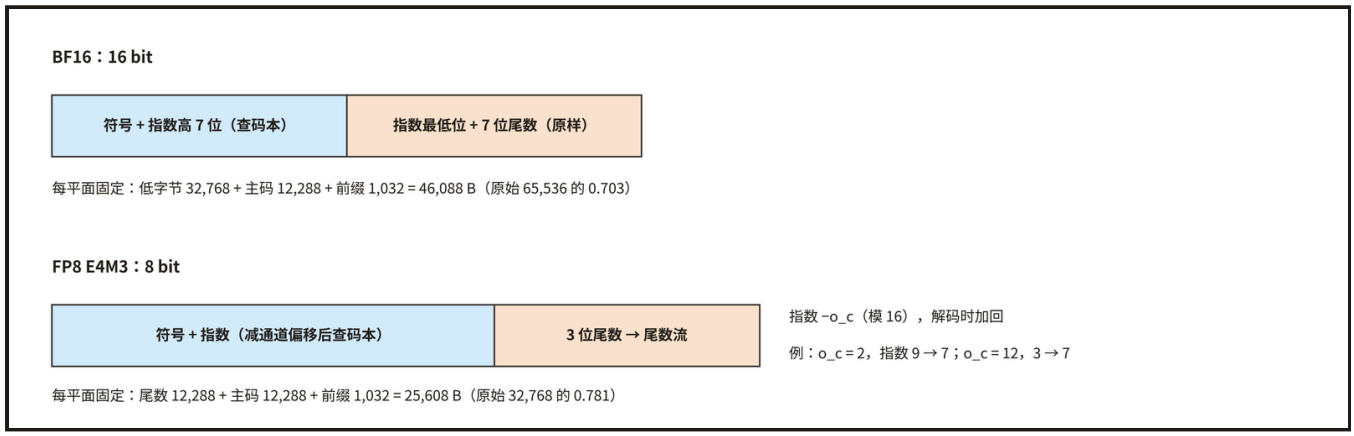


图 2-5 BF16 和 FP8 E4M3 一个标量怎么拆，FP8 的指数先按通道移位

vLLM 的 FP8 cache（`--kv-cache-dtype fp8`）每个标量 1 字节，E4M3：1 位符号、4 位指数、3 位尾数。codec 把它拆成两部分：

- **符号 + 指数（5 bit 符号 `sign << 4 | exp`）**：先按这个标量所在通道的偏移移位（下面），再像 BF16 的高字节一样查这个头的码本。通用表 `_GENERIC_FP8_ONLINE_CODEBOOK` 是 7, 23, 6, 22, 8, 24, …：E4M3 的指数偏置是 7，所以 7 是正、绝对值在 [1, 2)，23 = 16 + 7 是它的负数。
- **尾数（3 bit）**：所有标量的尾数打包成一条稠密的 3 bit 流，代替 BF16 的低字节流。每平面 ⌈32,768 × 3 / 8⌉ = 12,288 B。

例子的 FP8 chunk 是 3 MiB。固定部分每平面 12,288（尾数）+ 12,288（主码）+ 1,032（前缀）= 25,608 B，96 个平面加 8 KiB 头是 2,466,560 B，已是原始的 0.784。

问题：同一个头里各通道的量级差得远。一个头 128 个通道共用一张 15 项的表，而有的通道常年在 [4, 8)，有的在 [1/16, 1/8)，它们最常见的指数不一样，一张表顾不过来，多出来的都是转义。FP8 比 BF16 更怕这个：BF16 的高字节丢了指数最低位，一个值跨 4 倍的量级（63 是 [0.5, 2)），FP8 一个符号只跨 2 倍，同样的动态范围要多一倍的符号才盖得住。[推断]

做法：逐通道指数偏移。每个（层，K/V，头）每个通道 c 有一个偏移 o_c（0–15，`offset[L, 2, H, head_dim]`）：

- 编码：符号的指数减 o_c 再取模 16，符号位不动（`shift_fp8_symbols`）：`sign | ((exp - o_c) mod 16)`。码本统计和查找都用移位后的符号。
- 解码：查表得到移位后的符号，指数加回 o_c 再取模 16。

模 16 的减法是 16 个指数上的一个置换，每个值都换得回来，所以无损；移位后落到表外的值照样走 raw。例子：头的最常见指数是 7（[1, 2)），通道 a 常见指数 9（[4, 8)），o_a = 2，移位后是 7；通道 b 常见指数 3（[1/16, 1/8)），o_b = (3 − 7) mod 16 = 12，(3 − 12) mod 16 = 7。两个通道最常见的值都变成了符号 7，共用一个主码。通道 a 里少见的指数 1 会变成 (1 − 2) mod 16 = 15，多半走 raw，但仍然能解回来。

偏移怎么拟合（`attn_page.calibrate_fp8`，每个（层，K/V，头）单独做）：

1. 初值：每个通道的最常见指数减去整个头的最常见指数，取模 16（正负号的直方图合起来算）。
2. 交替 4 轮（`_FIT_ROUNDS = 4`）：按当前偏移移位后数频率、排出一张表；用 FragPack 的代价估计总位数（排名 0–6 不额外花，7–21 花 4 bit，22 以后花 36 bit）；再给每个通道单独挑一个让自己位数最少的偏移。
3. 保留估计位数最少的那一轮的表和偏移。

tier 直接用同一组偏移和同一张表的前 14 项，没有按自己的代价（次码 3 bit、raw 3 + 8 bit、只有 14 项）再拟合一遍。这样做是为了共用一份 artifact；对 tier 是不是最优，代码里没有说。［读码］［推断］

K 要跨位置标定：K 存的是 RoPE 之后的值，RoPE 把通道对（c, c + 64）按位置旋转，同一个通道在第 100K 个 token 上的量级和在抓取时的短位置上不一样。只用抓取位置拟合的 K 偏移，到长位置上就会不准。给 `calibrate.py` 加上 `--rope-theta θ --max-positions N` 后，K 用抓到的 K 重新旋转到 [0, N) 里均匀取的 16 个位置（`ROPE_CALIBRATION_POSITIONS = 16`，`rerotate_k`，NeoX `rotate_half` 的配对）再拟合；V 不受 RoPE 影响，照常拟合。

效果：在这个模型的 agent KV 上（tier record，同一份抓取，前 8 块标定、其余评估），按平面一张码本是 .900；每头码本加逐通道偏移是 .868；不加 RoPE 重旋转是 .867。RoPE 版本在抓取的短位置上略差一点，换来长位置上不失效；长位置上差多少这里没测。［实测］`codec_pareto_bench`，θ = 1e7、N = 262,144

3. GPU 上的编码和解码

编码和解码都是 TileLang 写的 kernel，在 cache server 进程里构造 `KVMajorChunkCodec` 时一次编译好（`warm_fused_online_kv_packer` 先空跑一遍），请求路径上不付 JIT。编码一批 8 个 chunk 是三次 launch，主机只在最后读一次长度；解码一批是一次 launch，主机什么都不等。

读码 `lmcache/v1/lossless_kv/tilelang_kernels.py compressed_kv.py`

3.1 编码：count、layout、compact 三次 launch，长度最后读一次

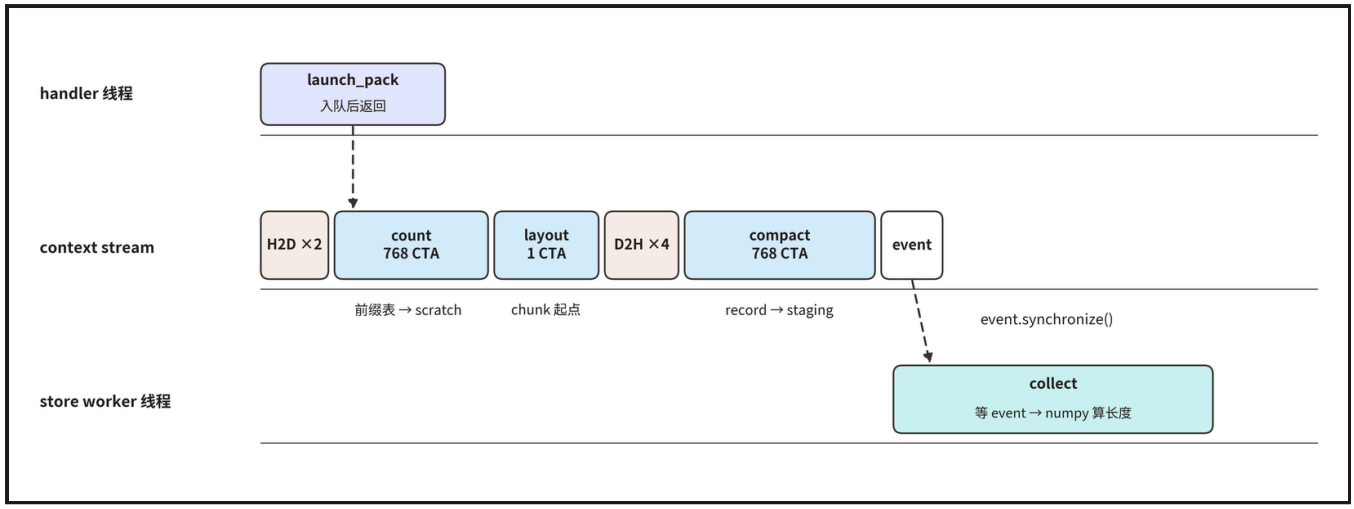


图 3-1 编码一批 8 个 chunk 的三次 launch

解决什么：变长 record 要背靠背写进同一块 staging，每个 record 写在哪取决于前面所有 record 有多长；而长度要先数完转义才知道。如果每数完一批就回主机算偏移，再发下一个 kernel，编码路径上就多一次 GPU 同步。

做法：`FusedOnlineKVPacker.launch_pack`（compressed_kv.py，输入：引擎块号、staging、stream；输出：`PendingPack`，含一个 CUDA event 和这次用的 pinned 主机缓冲）把下面这串全部排在同一条 stream 上，中间不回主机：

1. 两次 H2D：块号和 slot 号。
2. **count**（grid = 8 个 chunk × 96 个平面 = 768 个 CTA，每个 256 线程）：一个 CTA 负责一个平面，先把这个平面每个头的 256 项查找表装进 shared memory（例子 2 × 256 B），8 个 warp 各数若干 tile，一个 lane 一次读 8 个标量，FP8 先按通道移位（3.2 节末），再查自己那个头的表；数完做一次 block scan，写出这个平面的两张前缀表和总转义数、总 raw 数。
3. **layout**（1 个 CTA，1024 线程 = 32 个 warp）：一个 warp 负责一个 chunk，把 96 个平面长度加起来，判断 raw（2.4 节），再在 32 个 warp 之间扫一遍得到每个 chunk 在 staging 里的起点。1024 是一个 CTA 的线程上限，32 个 warp 正好让一次 lane 级扫描覆盖 32 个 chunk；批大于 32 时按 32 一段串行扫，游标留在寄存器里。全局前缀只能在一个 CTA 里做，这是它只有一个 CTA 的原因。
4. 四次 D2H（非阻塞，进 pinned 缓冲）：每平面总转义数、总 raw 数、每 chunk 的 overflow 标志和符号宽度。
5. **compact**（grid = 8 × 96，每个 256 线程，`_COMPACT_MIN_BLOCKS = 5`，每 SM 5 个 CTA）：重新读一遍 KV，一个 CTA 写一个平面：低字节和 3 bit 主码先进共享内存再按对齐的字写出，raw 高字节直接写进 record，次码按 count 留下的前缀表定位后原子 OR 进共享的位流。平面 0 的 CTA 顺带序列化 record 头。overflow 的 chunk 每个 CTA 原样拷自己那个平面。5 个 CTA/SM 是 48 寄存器/线程不溢出的上限，代码注释说 4 和 6 在 Qwen3-8B 上都更慢。

然后记一个 event。`collect(pending)` 在这个 event 上 `synchronize()`（等 event，不等整条 stream），读 pinned 缓冲，用 numpy 一次算出每个 record 的模式、起点和长度（2.4 节的公式），再和设备算的总长对一

遍，不一致就 `RuntimeError`。用 numpy 而不是 Python 循环，是因为 8 × 96 = 768 行的循环会在每次打包后占着 GIL，和同进程的 handler 线程抢。［读码］

launch 和 collect 分开：`launch_pack` 和 `collect` 可以在不同线程调用，每次打包有自己的一套 pinned 主机缓冲（`_PackHostBuffers`，用完放回空闲列表），所以 handler 线程 launch 完就走，store worker 线程去 collect（4.2 节）。这是 37c499d 拆的。[5]

3.2 解码：一个 warp 一个 tile，PRMT 查表，128 bit 直接写进块

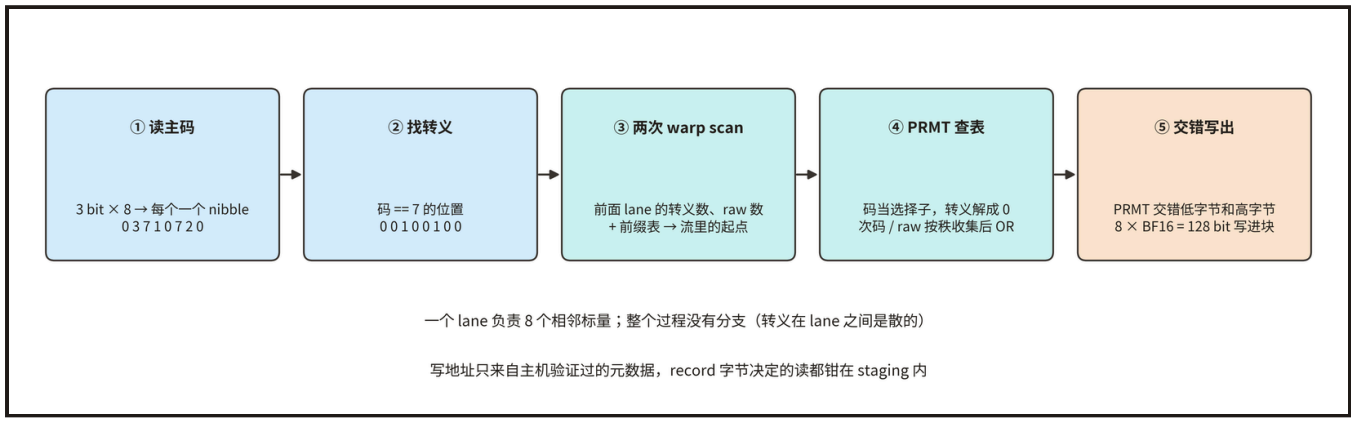


图 3-2 一个 lane 怎么解出自己的 8 个标量

解决什么：retrieve 时要把 record 还原进 vLLM 的分页块。如果先解到连续缓冲再拷进块，多出的一次读写比解码本身还贵；解码还要在 lane 之间没有分支，因为转义散在各处。

grid：`_build_load` 的 grid 是（chunk 数，平面数，每平面 CTA 数），每个 256 线程。一个 CTA 读一次平面描述符，然后解 `_LOAD_TILE_GROUPS = 32` 组、每组 8 个 tile，8 个 warp 一个 warp 一个 tile。例子一个平面 128 个 tile = 16 组，不到 32 组，所以每平面 1 个 CTA，一批 768 个 CTA。32 组是在 Qwen3-8B 的平面（512 个 tile）上和 16、64 比出来最快的。

一个 lane 做什么：

1. 读自己 8 个标量的 3 bit 主码（24 bit），把每个码摊成一个 nibble。码等于 7 的位置就是转义，用位运算一次标出 8 个 nibble 的转义标志。
2. 两次 warp 内前缀和：前面的 lane 一共有多少个转义、多少个 raw；加上前缀表里这个 tile 之前的累计数，就是自己在次码流和 raw 流里的起点。不用逐个标量投票。
3. 查表：码本在启动时由 `pack_load_codebooks` 重排成每（平面，头）16 字节，lane 只读自己那个头的一行，字节 0–6 是主码的 7 个值、8–14 是次码的 7 个值，7 和 15 是 0。nibble 当作 PRMT（字节置换指令）的选择子，一次 PRMT 解出 4 个标量的高字节；转义位置选到的是 0。
4. 转义的标量从次码流按秩取值：`_rank_selector` 把转义标志变成选择子，第 i 个 nibble 是「i 之前有几个转义」，再用 PRMT 从次码字节里按顺序收集；raw 字节同样从一个钳住边界的 8 字节窗口里收。结果 OR 进第 3 步的结果。

5. 再两次 PRMT 把低字节和高字节交错成 BF16，`head_dim % 8 == 0` 时一次 128 bit 写进目标块。 FP8 没有低字节：先把解出的 5 bit 符号的指数按通道加回偏移，再和 3 bit 尾数拼成字节。

每头查表和 FP8 移位：编码的 count、compact 和解码 kernel 用同一套办法找每个 lane 的表和偏移。［读码］[8]

- 一个 lane 的 8 个标量总在同一个头里（构造时要求 `head_dim % 8 == 0`），所以一个 lane 只用一行码本。头号 = `(起点 // head_dim) % 头数`，通道 = `起点 % head_dim`。
- `pack_fp8_shifts` 把一个 lane 8 个通道的偏移（模 16，编码用 −o、解码用 +o）压进一个 uint32：通道 c < 4 在第 c 个字节的低 4 位，通道 4 + c 在高 4 位。lane 的 8 个 FP8 字节是两个 uint32，正好一个取低半、一个取高半。
- `_shift_fp8_lane` 一次处理 4 个字节：取出每个字节的指数域（编码时是存储字节的第 3–6 位，解码时是 5 bit 符号的第 0–3 位），加上移位值再按域掩码截回。每个字节里指数加移位小于 32，进位只会冒出指数域，不会进到下一个字节；冒出来的那一位被掩码丢掉，就是模 16。符号位和尾数从原字节保留。所以 8 个标量的移位是 2 次加法加几次位运算，不用逐字节循环。
- 8 个 warp 每一步让一个 lane 前进 8 个 tile = 2,048 个标量。一行 token（头数 × head_dim）能整除 2,048 时，lane 每一步都落在同一个头的同一组通道上，表和移位字只在开头读一次（`_lane_rows_fixed`）；不能整除时每一步重算。例子一行 2 × 128 = 256，整除。例如 warp 1 的 lane 5 起点是 1 × 256 + 5 × 8 = 296，头 (296 // 128) % 2 = 0，通道 296 % 128 = 40–47。

代价：码表、移位字多读一些，FP8 多两次加法。在这个模型的 agent KV 上，256 块一批的吞吐 BF16 编码 518 → 499 GB/s、解码 1224 → 1191 GB/s，FP8 编码 265 → 253、解码 655 → 623，都慢 3–5%，结果逐位相同。［实测］`codec_pareto_bench`，改动前后同一份抓取、同一张 H800

健壮性：kernel 关掉了 TileLang 的逐次访问越界检查。每个全局写地址只由主机验证过的元数据（目标块号、平面范围、grid 下标）算出，不由 record 字节决定；由 record 字节决定地址的读都钳在 staging 范围内。所以一个损坏的 record 只会把错误的值写进它自己的目标块。

3.3 KVRestorePlan：解码的元数据只上传一次

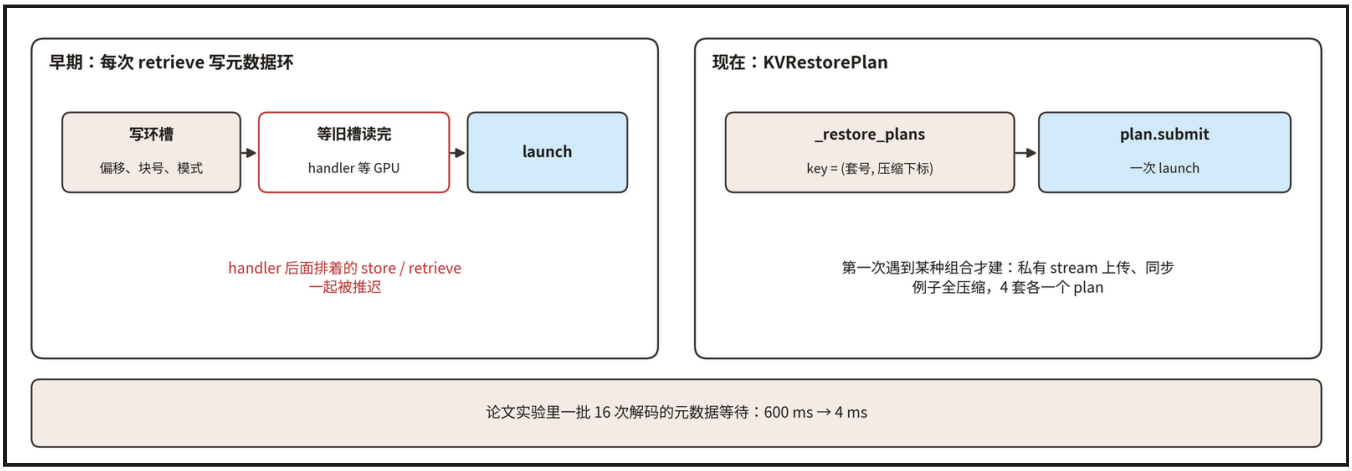


图 3-3 旧的元数据环和现在的 restore plan

问题：解码 kernel 需要每个 record 在 staging 里的偏移、目标块号、模式（raw 或压缩）。早期版本每次 retrieve 把这些写进一个设备上的元数据环，写之前要确认环上的旧槽已经被之前的解码读完，也就是 handler 线程要等 GPU。而 handler 线程前面排着别的 store 和 retrieve，等一次就把后面都推迟：论文实验里一批 16 次解码的元数据等待是 600 ms，修完是 4 ms。[4]

做法：`FusedCompressedKVDecoder.plan`（输入：staging、每个 record 的偏移、目标块号、模式；输出：`KVRestorePlan`）在一条私有 stream 上把元数据一次传上去，同步这条私有 stream，返回一个不可变的 plan；之后 `plan.submit(stream)` 只是一次 launch，主机不做别的。staging 的布局是固定的（每个 record 一个原始大小的槽），所以元数据只取决于「这批第几个是压缩的」：`KVMajorChunkCodec` 按（buffer 套号，压缩 record 的下标元组）缓存 plan。例子里全压缩的 8 个 chunk 只有一种组合，四套 buffer 共 4 个 plan，第一次 retrieve 建好以后都复用。

两种 plan：`plan` 的目标块固定；`plan_dynamic` 不带目标块，每次 `submit(stream, blocks)` 传一个设备上的块号张量。直接解进引擎块时块号每次不同，用后一种，key 是（套号，批大小）。`plan` 还支持一个 record 解进多个块（fanout，同一个偏移出现多次），编译时两种变体都编好。

3.4 直接读写引擎块：vLLM 0.30 的层张量首尾相接

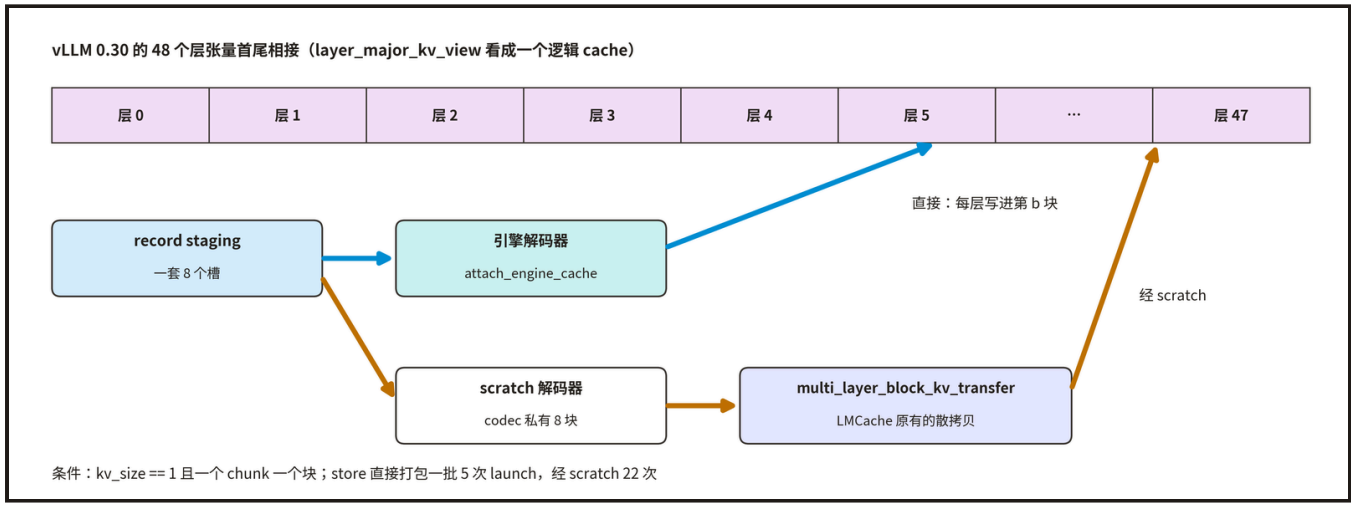


图 3-4 直接读写引擎块和经 scratch 的两条路

两条路：

- **经 scratch**：codec 自己有一块私有的 cross-layer cache（`_encode_scratch` 和 `_decode_scratch`，各 8 个块），编码先把 LMCache 暂存的 chunk 拷进 scratch 再编，解码先解进 scratch 再用 LMCache 原有的 `multi_layer_block_kv_transfer` 散进分页块。
- **直接**：`LosslessKVTransfer._attach_engine_cache` 在注册时检查两件事：`kv_size == 1`（fused K/V）并且一个 chunk 正好一个块。满足就用 `layer_major_kv_view` 把 48 个层张量看成一个逻辑 cache（要求它们在同一块分配里首尾相接，否则返回 None 走 scratch），然后 `attach_engine_cache` 在这个 view 上再编一个解码器和一个打包器。打包器只在变长模式下编，固定容量模式要原地补齐，需要原始大小的缓冲。

例子满足两个条件，日志里会打 `Lossless KV retrieve decodes straight into engine blocks`。直接打包时一批 5 次 launch，原来经 scratch 的路径是 22 次，也少扫两遍原始字节。[5]

为什么 kernel 能同时支持两种布局：编码和解码 kernel 编译时带一个 `layer_major` 参数，决定把 KV 当作块在外（`[块, 层, …]`，vLLM 的 cross-layer 布局和 codec 自己的 scratch）还是层在外（`[层, 块, …]`，vLLM 0.30 每层一个张量）。地址用 int64 算，因为层在外时跨层的偏移是整个层张量的大小，例子里一层是 54 GiB / 48 ≈ 1.1 GiB，乘上层号会超过 2^31。

4. tier mode 的 store：handler 只 launch，长度由 store worker 读

tier mode 改 store 改的是「谁在什么时候知道对象多大」。原来对象大小在注册时就定了，handler 线程预留、拷贝、提交都不用等 GPU；现在 record 长度编码完才知道，读它就要同步 GPU，这一步被挪到每个 GPU 一个的 store worker 线程上，handler 线程只排 kernel 就返回。

读码 `lmcache/v1/multiprocess/lossless_kv_transfer.py`

4.1 原来的 store：注册时定好对象大小，提交挂在 stream 回调上

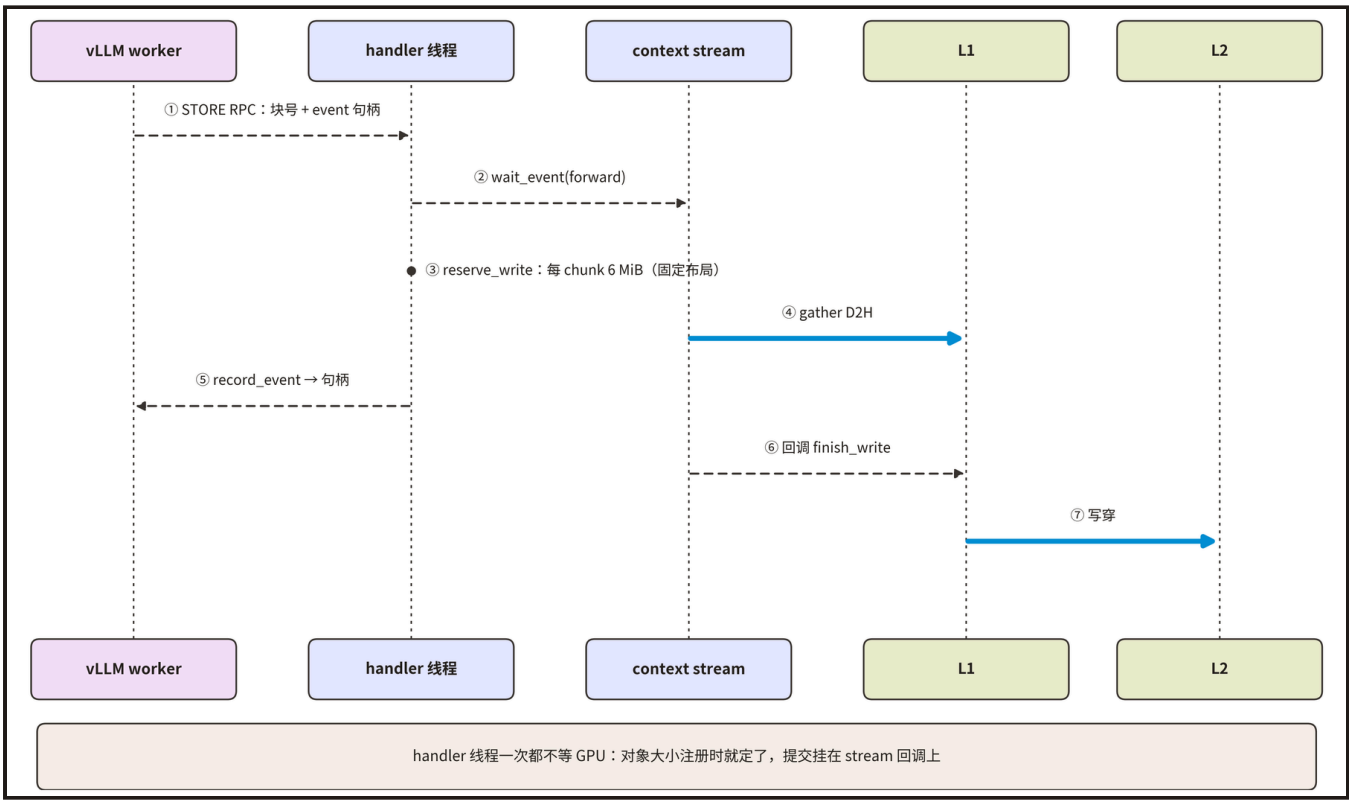


图 4-1 LMCache 原来的一次 store

```
vLLM worker  wait_for_save()  记一个 event，batched_submit_store_requests → STORE RP
cache server  GPU 的 affinity 线程
└── store → store_with_chunk_mask(key, instance_id, gpu_block_ids, event_ipc_handl
    ├── resolve_obj_keys          token 区间 → 每个 chunk 一个 ObjectKey（每个 rank 各
    ├── 块号不够覆盖全部 chunk      记 event 返回 False，整个 store 跳过（防越界读）
    ├── import_event + wait_event  context stream 等 vLLM 这一步的 forward
    ├── reserve_write(keys, layout_desc)   每个 chunk 一个 L1 对象，大小 = 注册时的固定布
    ├── transfer_kv_per_object_group(D2H)  分页块 gather 进 pinned 对象，排在 context
    ├── prepare_write_completion
    ├── record_event(event)         这个 event 过了，vLLM 就能复用这些块
    └── submit_callback_to_stream("finish_write_by_owner")   拷贝完成时在回调里提交，对
L1 → L2  StoreController 异步写穿到 fs_native（O_DIRECT）
```

返回给 worker 的是第二个 event 的 IPC 句柄，vLLM 等过它才复用这些块。整个 store 是全有或全无：任一 chunk 拷贝失败，整批都不提交，之后 retrieve 未命中、vLLM 重算。

这条路径上 handler 线程从不同步 GPU：对象大小早就知道，提交也挂在 stream 回调上。codec 打破的是第一条。

4.2 codec store 的调用链：两条线程、两条 stream

`RecordStoreWorker`（lossless_kv_transfer.py，每个 cache context 一个）有一块 GPU 上的缓冲池、一个任务队列和一个名为 `lossless-kv-store` 的线程；handler 往池里打包、把任务放进队列，线程按提交顺序把 record 存进 L1。下面的 worker 指它。

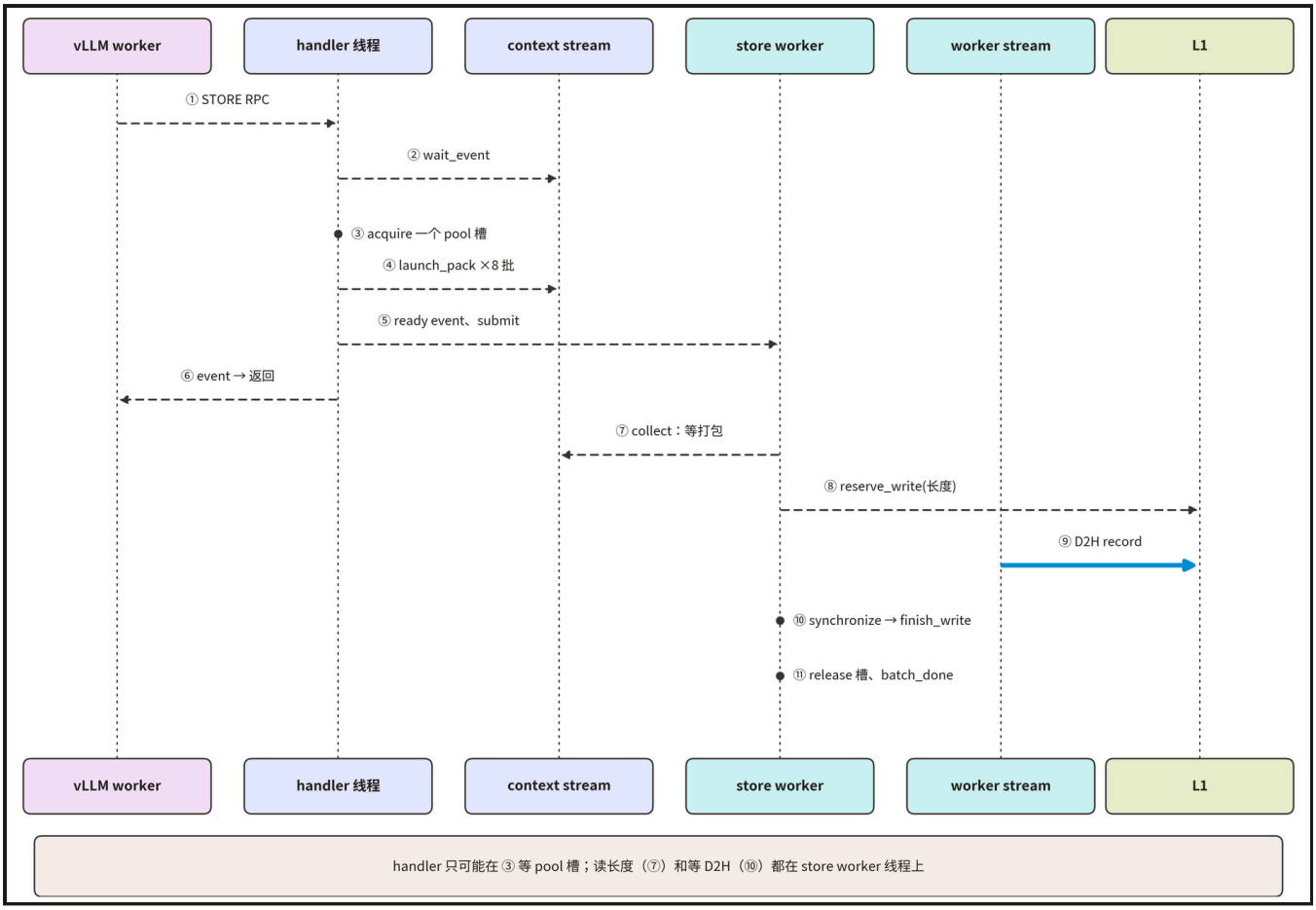


图 4-2 codec store 时谁等谁

```
cache server  GPU 的 affinity 线程（handler）
└── store_with_chunk_mask
    ├── import_event + wait_event          同 4.1
    ├── tally = StoreTally(_codec_store_reporter(...))   这次 store 的计数器，4.4 节
    ├── LosslessKVTransfer.submit_store(块号, keys, skip_mask, tally)
    │   ├── storage.l1_contains(keys)     已在 L1 的 chunk 直接跳过，不编码
    │   └── 每 8 个 chunk 一批 → _pack_engine_window
    │       ├── worker.acquire(1)         取一个 pool 槽；pool 满就在这里阻塞
    │       ├── codec.encode_engine_blocks → packer.launch_pack   3.1 节，排在 conte
    │       ├── ready = Event().record(context stream)
    │       ├── tally.add_batch()
    │       └── worker.submit(RecordStoreJob)    进队列，槽从此归 worker
    ├── record_event(event)                vLLM 可以复用块了（打包已读完它们）
    ├── tally.seal(ok)
    └── 返回 (event 句柄, ok, 每个 chunk 是否已入队)
lossless-kv-store 线程（worker，自己的 stream）
└── _store(job)
    ├── stream.wait_event(job.ready)
    ├── job.records(...) → codec.engine_records → packer.collect   等打包的 event，读
    ├── 按长度分组 → storage.reserve_write(keys, codec_byte_layout(长度))
    ├── lmcache_memcpy_async_d2h(record, L1 对象)   每个 record 拷进自己的对象
    ├── prepare_write_completion → stream.synchronize() → finish_write_by_owner
    ├── release(job.slots)                 槽还给 pool
    └── job.on_done(stored, ok) → tally.batch_done
```

谁等谁：

- handler 线程只在 `acquire` 上可能等，等的是 pool 有空槽（4.3 节）。它不读长度，所以后面排着的 retrieve 不会被 store 的同步拖住。
- worker 线程等两次：`collect` 等打包 event（读长度），提交前 `synchronize()` 等 D2H 拷完（L1 对象必须写满才能对 lookup 可见）。它有自己的 stream，所以这两次等待不影响 context stream 上的 retrieve。
- vLLM 等 handler 返回的 event。这个 event 记在打包之后，打包读完了引擎块，所以块可以立刻复用，不用等 worker 把 record 拷出去。

块号从主机来：`_pack_engine_window` 用的是主机上的块号列表 `block_ids_host[0]`，launch 时 H2D 上去，不用 LMCache 已经暂存在 GPU 上的块号张量，因为打包器要按自己的批切片。

不能直接打包时（不是 fused K/V，或一个 chunk 跨多个块）：handler 用 `launch_paged_kv_copy` 把每个 chunk gather 进 pool 的一个原始大小槽（连续的 chunk 合成一次拷贝，每次 kernel 最多 4 个 chunk），记 ready event；worker 在自己的 stream 上调 `codec.encode`，它内部同步读长度后把 record 原地写回槽的开头。

一个 step 的量：vLLM 默认一步 8192 token 的预算，prefill 满载时一个 rank 一步产生 64 个 chunk，即 8 批、8 次 `acquire`、8 个 job。每批 48 MiB 原始数据，编出约 34 MiB 的 record。

4.3 store pool：默认 128 个 chunk，够两个 step

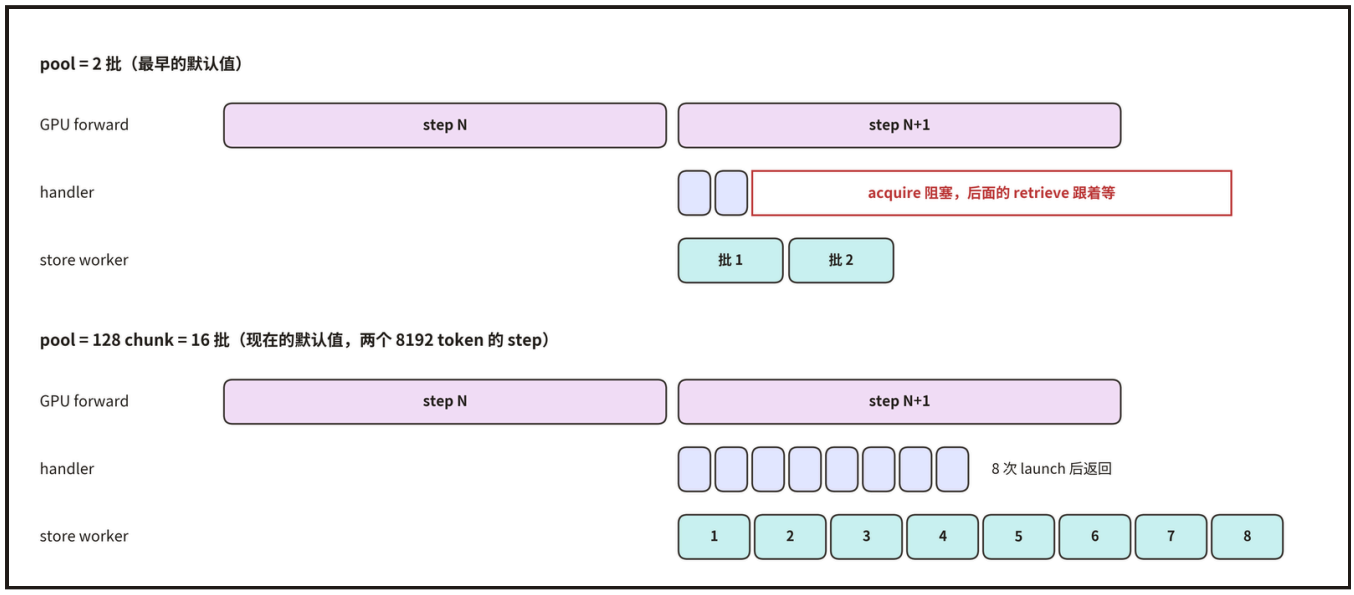


图 4-3 pool 太小时 handler 怎么被卡住

`RecordStoreWorker` 在 GPU 上分配一整块 uint8 pool，按槽切。槽的大小和个数取决于走哪条路：

路径	槽大小	槽数（默认 `--lossless-kv-store-pool 128`）	显存
直接打包（例子）	一批：8 × 6 MiB = 48 MiB	128 / 8 = 16	768 MiB
gather 后编码	一个 chunk：6 MiB	128	768 MiB

为什么是 128：一批要等产生它的那一步 forward 在 GPU 上跑完才能编码，worker 再读长度、拷出、提交，才释放槽。最早的默认值是两批，结果 handler 线程大部分时间卡在 `acquire` 上，排在它后面的 retrieve 也跟着晚。128 个 chunk 是两个 8192 token 的 step，一步的 store 永远不用等上一步的 worker 收尾。[3]

pool 满怎么办：`acquire` 用条件变量等，不丢 store。编码跟不上时 store 变慢，handler 线程被拖住，这是背压；比起丢掉 store、之后重算，这样更便宜。gather 路径一批最多取 `min(8, 槽数)` 个，所以 pool 小于一批也不会死等。

显存从哪来：cache server 是 vLLM 以外的进程，它的 pool、codec 的 scratch 和 staging 用的是 vLLM 按 `--kv-cache-memory-bytes` 分完 KV 之后剩下的显存。例子里一个 rank 上大约是：pool 768 MiB，解码 staging 4 套 × 8 × 6 MiB = 192 MiB，打包 staging 48 MiB，编码和解码 scratch 各 48 MiB，合计约 1.1 GiB。[推断]

4.4 StoreTally：一次 store 只上报一次

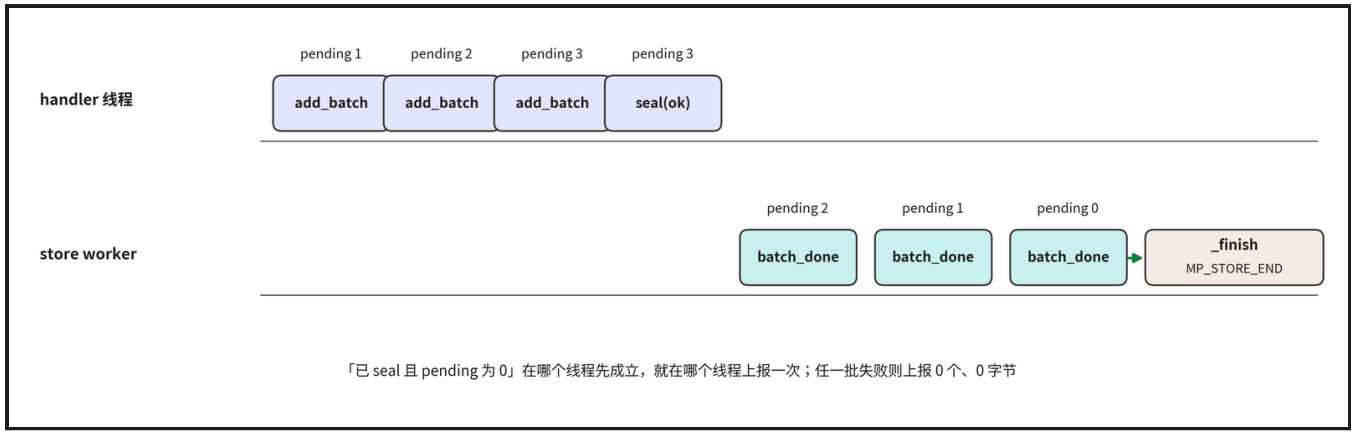


图 4-4 一次 store 的几个批怎么汇成一次上报

问题：LMCache 的统计和事件（`MP_STORE_END`：存了几个对象、多少字节）按请求算，每次 store 上报一次。原来上报在 handler 线程里、挂在 stream 上。现在一次 store 拆成若干批，它们在 worker 线程上陆续完成，最后一批完成时 handler 早就返回了；批数也可能是 0（全部已在 L1）。

做法：`StoreTally` 是一个带锁的计数器。

- handler 每提交一批前 `add_batch()`（pending + 1），提交完所有批后 `seal(ok)`。
- worker 每完成一批调 `batch_done(stored, ok)`：pending − 1，累加对象数和字节数。
- 「已 seal 且 pending 为 0」第一次成立时调 `_finish()`，不管这发生在哪个线程。没有批时就是 `seal` 自己触发。
- 任何一批抛异常，或 handler 提交时出错，`ok` 变成 False，上报 0 个、0 字节。

`_codec_store_reporter` 生成的回调直接 `publish` 事件，不走 `publish_on_stream`，因为它已经在 worker 线程上、拷贝已经同步过了。

边界：上报 0 不等于什么都没存。已经提交的批在 L1 里照样可以命中；上报只是统计口径。handler 返回的每 chunk 标志表示「已入队」而不是「已提交」，所以 worker 提交之前的那一小段时间里，同一前缀的 lookup 会未命中，最多导致一次多余的重算。

4.5 L1 和 L2 按 record 长度存

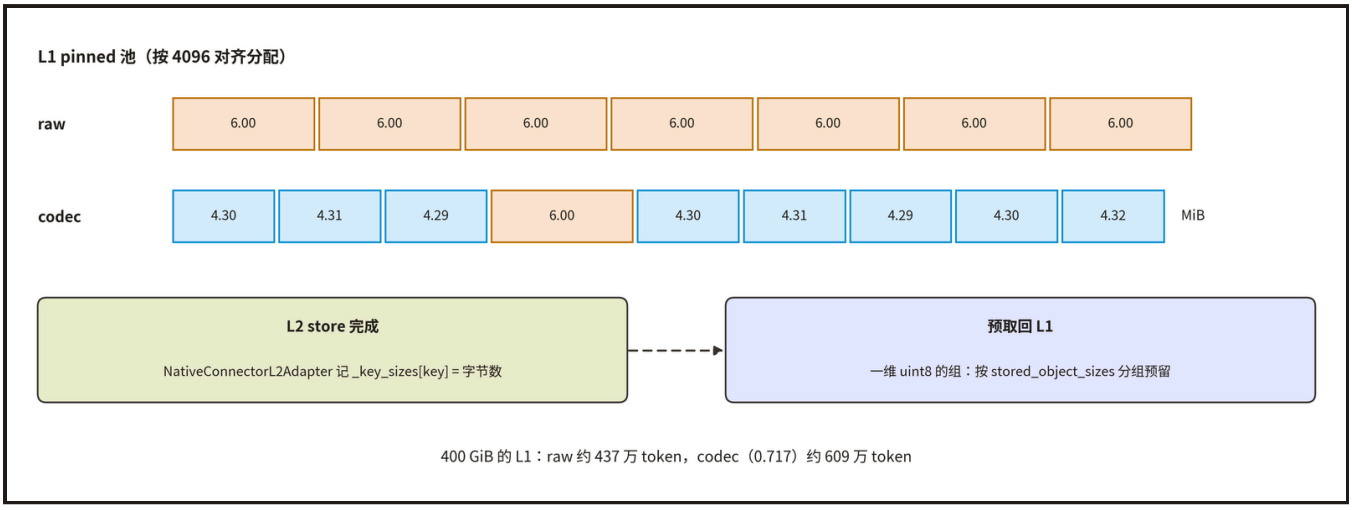


图 4-5 L1 按长度存 record，L2 按记下的大小预取

为什么必须改存储层：如果 L1 还按原始 chunk 大小分配对象，4.3 MiB 的 record 放进 6 MiB 的对象，剩下的 1.7 MiB 白占着，L1 能装的 chunk 数一个不多，只省了 PCIe 字节。所以对象大小要等于 record 长度。

改了四处（加起来约 160 行）：［读码］ `lmcache/v1/distributed/`

1. **注册的布局**：开 codec 时 `register_kv_cache` 不再用 `get_layout_desc` 算出的 KV 张量布局，而是 `LosslessKVTransfer.layout_desc`：一个一维 uint8，长度 = 每对象 chunk 数 × `capacity_bytes`（变长模式下就是原始大小）。它只是个上限。
2. **按长度预留**：worker 线程把一批 record 按长度分组，每种长度调一次 `reserve_write(keys, codec_byte_layout(长度))`，长度相同的共用一次调用。L1 的分配器按 4096 字节对齐，record 本来就是 4 KiB 的倍数，不浪费。预留失败（L1 满、key 已存在）的 key 拿不到对象，就不拷不存。
3. **先查再编**：`L1Manager.contains` 和 `StorageManager.l1_contains` 让 handler 在编码前跳过已经在 L1 的 chunk（`reserve_write` 本来就会拒绝它们，提前跳过省一次编码）。查和预留之间可能有别的线程写入，所以结果可能过期，过期时只是白编一次，第 2 步照样拒绝。
4. **L2 预取**：L2 的 `fs_native` 原样写对象，record 已经 4 KiB 对齐，O_DIRECT 照常可用。从 L2 预取回 L1 要一块和存储时一样大的缓冲，而这个大小只有 L2 知道。`L2AdapterInterface.stored_object_sizes` 是新加的接口，`NativeConnectorL2Adapter` 在 store 完成时记下每个 key 的大小（`_key_sizes`）。`PrefetchController._stored_byte_sizes` 对「布局是一维 uint8」的组（`_is_byte_layout`）问 L2 要大小，`_reserve_l1_cells` 按大小分组预留。普通的 KV 布局只有一种大小，不受影响。另外 `StoreController` 统计提交字节数时原来是「第一个对象大小 × 个数」，改成逐个相加。

容量：400 GiB 的 L1 装原始 chunk 是 400 GiB / 6 MiB = 68,266 个对象，两个 rank 合起来 34,133 个 128 token 的 chunk，约 437 万 token；装 0.717 的 record 是约 609 万 token，多 1.39×。L2 同理。

5. tier mode 的 retrieve：长度来自主机，拷贝跑在解码前面

retrieve 路径上完全不读 GPU 回主机：每个 record 的长度就是它在 L1 里那个对象的大小，主机本来就知道。handler 线程只把拷贝和解码排进 stream 就返回，拷贝走一条单独的 fill stream，在四套缓冲之间轮转，所以后面几批的 PCIe 拷贝跑在前面几批的解码之前。

读码 `lmcache/v1/multiprocess/lossless_kv_transfer.py` `lmcache/v1/lossless_kv/chunk_codec.py`

5.1 retrieve 的调用链

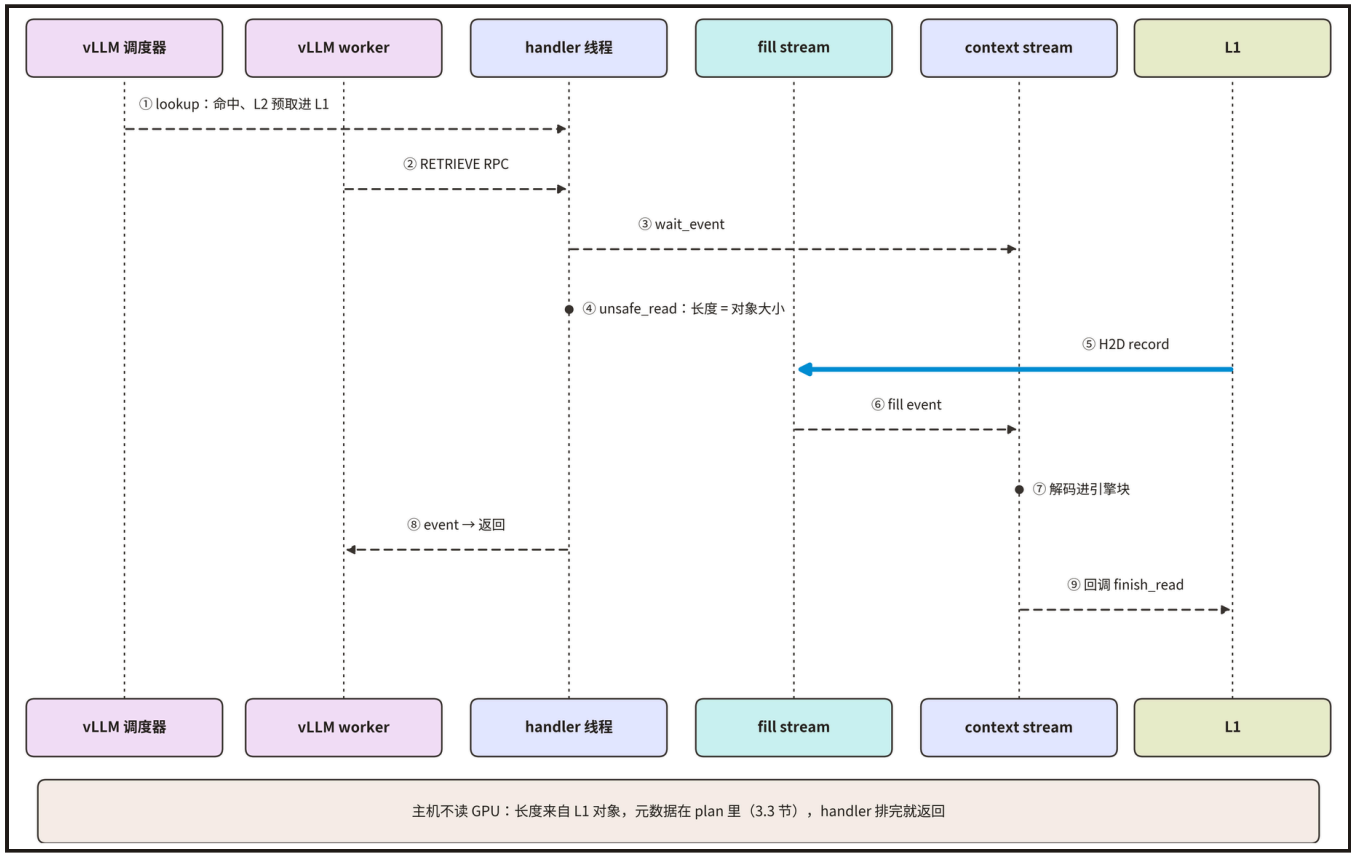


图 5-1 一次命中 L1 的 retrieve

```
vLLM 调度器  get_num_new_matched_tokens → lookup RPC    命中前缀；只在 L2 的 chunk 由 Pr
vLLM worker  start_load_kv()  记 event，batched_submit_retrieve_requests → RETRIEVE
cache server  GPU 的 affinity 线程（handler）
└── retrieve
    ├── import_event + wait_event                    context stream 等 vLLM 这一步之前
    ├── prepare_read_completion(keys)                对象在读完之前不会被逐出
    ├── unsafe_read(keys) → window_objs              L1 里的 MemoryObj，每个 chunk 一个
    ├── LosslessKVTransfer.retrieve(块号, [None]*skip + window_objs, skip_first_n_t
    │   ├── first = max(第一个非 None, skip_first_n_tokens // 128)    整块落在 APC 前缀
    │   ├── engine_blocks = 块号 pinned → GPU（非阻塞，在 context stream 上）
    │   └── 每 8 个 chunk 一批
    │       ├── sizes = [obj.get_size()]             长度：主机上的对象大小
    │       ├── buffers = codec.record_buffers(n, fill_stream)    轮到下一套，等它上次的
    │       ├── fill_stream 上 lmcache_memcpy_async_h2d(obj, buf[:size])    PCIe 拷贝
    │       ├── codec.decode_records_to_engine(sizes, 块号片段, context stream)    直
    │       └── 返回 False（有 raw 或部分跳过）→ codec.decode_records → launch_paged_k
    ├── record_event(event)                          vLLM 在 forward 前等它
    └── submit_callback_to_stream("finish_read_by_owner")    读锁在拷贝和解码之后释放
```

读锁什么时候放：`finish_read_by_owner` 挂在 context stream 的回调上。fill stream 上的拷贝读的是 L1 对象，而每次解码前 context stream 都等了这批的 fill event（`_order_decode`），回调排在所有解码之后，所以回调执行时拷贝一定已经完成，对象可以安全地被逐出。

失败：一批里出现 None 跟在 record 后面（前缀中间缺了一块），或者长度超出 codec 的范围，抛 `ValueError`；handler 的 `except` 把 `retrieve_succeeded` 置 False，vLLM 把这些块标成加载失败并重算。

5.2 四套 record buffer：拷贝不排在解码后面

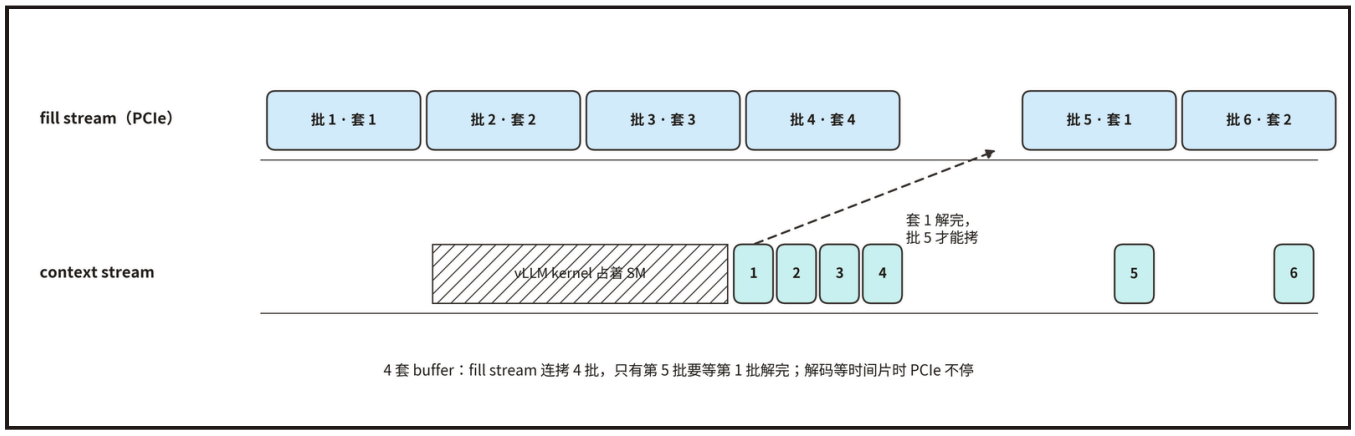


图 5-2 四套 buffer 轮转时 fill stream 和 context stream 的时间线

问题：如果拷贝和解码在同一条 stream 上，第 k + 1 批的拷贝要排在第 k 批的解码后面；而解码 kernel 要和 vLLM 的 forward 抢 SM，排队等时间片时，PCIe 就空着。

做法：`KVMajorChunkCodec` 有 `decode_sets = 4` 套 record buffer，每套 8 个原始大小的槽（例子里一套 48 MiB，四套 192 MiB），一个原始大小的槽放得下任意长度的 record。

1. `record_buffers(count, fill_stream)` 先在解码 stream 上记一个 event，表示「上一套到此为止的读者都排进去了」，存为那一套的 `_set_free`；然后换到下一套，让 fill stream 等这一套上次的 `_set_free`。
2. handler 在 fill stream 上把 record 拷进来。
3. `decode_records` / `decode_records_to_engine` 先 `_order_decode`：在 fill stream 上记一个 event，让解码 stream 等它；解码共用 scratch，所以换 stream 时还要等上一个解码 stream。

这样 fill stream 可以连着拷 4 批，只有第 5 批要等第 1 批解完。这是 37b3ca6 加的。[6]

为什么是 4：代码没有写理由。按量级估，一批 8 个 record 约 36 MB，PCIe 5.0 x16 上约 0.7 ms；解码按 1229 GB/s（论文的分页恢复速度）只要约 0.04 ms，但它要等 vLLM 的 kernel 让出 SM。4 套足够让 PCIe 连续工作几毫秒，而每多一套就多 48 MiB 显存。［推断］

5.3 什么时候直接解进块，什么时候走 scratch

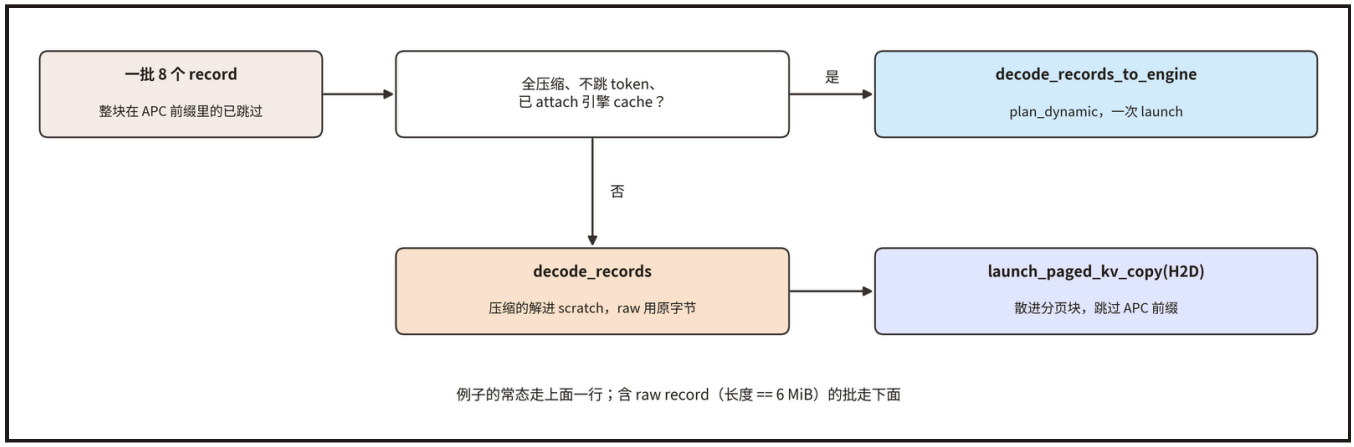


图 5-3 每一批 retrieve 走哪条路

这批的情况	走哪条路	理由
全是压缩 record，且不跳过 token	`decode_records_to_engine`：按（套号，批大小）取 `plan_dynamic` 的 plan，`submit(stream, 块号片段)`	例子里的常态，一次 launch
含 raw record（长度 == 6 MiB）	`decode_records`：只对压缩的下标建 plan 解进 scratch，raw 的直接用 staging 里的原样字节，再 `launch_paged_kv_copy` 散进块	引擎直写 plan 假定每个都是压缩的
第一个 chunk 只部分跳过	同上，散的时候跳过前面的 token	引擎直写一次写整块

这批的情况	走哪条路	理由
没有 attach 引擎 cache	同上	3.4 节的条件不满足

APC 跳过：`skip_first_n_tokens` 是 vLLM 自己 GPU 前缀缓存（APC）已经有的 token，不能覆盖（别的请求可能正在读那些块）。chunk 和块都是 128 token，vLLM 的前缀缓存也按块命中，所以跳过的 token 数在例子里总是 128 的倍数，整块落在跳过范围里的 chunk 在循环开始前就被 `first` 跳过了，「部分跳过」这一行只在 chunk 比块大的配置里出现。

固定容量模式：所有长度都等于容量类，没有 raw，全部按压缩 record 解。

6. pool mode：FragPack attention backend

pool mode 让 vLLM 的 GPU 池存压缩 KV，不改 vLLM 源码：LMCache 的 `pyproject.toml` 声明一个 `vllm.general_plugins` 入口，vLLM 启动时调用它，把 `FragPackAttentionBackend` 注册成 CUSTOM backend。vLLM 照常分配块、建块表、调度请求，只是每个块每层的那一页里放的是 FragPack record，读写都经过 backend 自己的表。例子里同样 54 GiB 的 KV 预算能放 1.60M token，raw 是 1.18M。［读码］ `lmcache/v1/lossless_kv/fragpack/`

6.1 怎么接进 vLLM：插件入口、KV spec 和一组限制

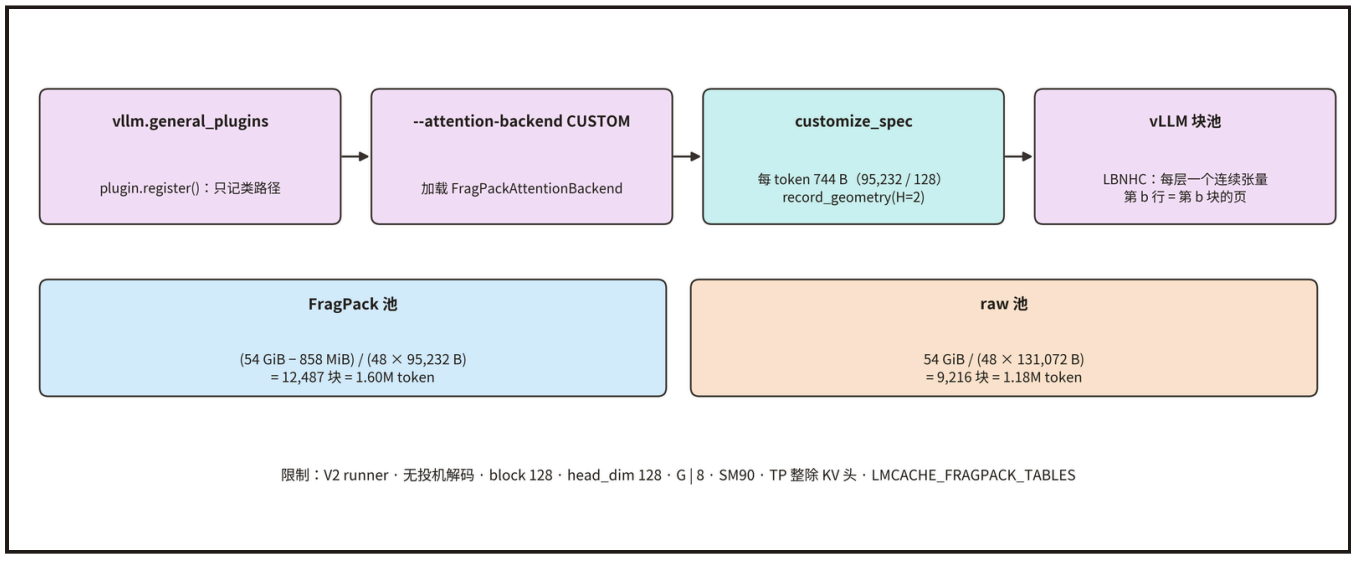


图 6-1 vLLM 启动时 FragPack 在哪几处接进去

1. **注册**：`plugin.register()` 只调一次 `register_backend(AttentionBackendEnum.CUSTOM, "…backend.FragPackAttentionBackend")`，记一个类路径，不 import 任何 CUDA 扩展，所以装了 LMCache 但不用 FragPack 的 vLLM 不受影响。只有 `--attention-backend CUSTOM` 才会真正加载它。harness 里

LMCache 是源码 checkout、不是 pip 装的，所以额外放了一个带 `entry_points.txt` 的 dist-info 目录到 `PYTHONPATH`（`benchmarks/lossless_kv/plugin_metadata/`），让 vLLM 找得到入口。

2. **定块大小**：vLLM 按每个 attention 层的 `KVCacheSpec` 算一个块占多少字节（《vLLM 的 KV 管理：spec、分组、布局、块池和 TP / PP / DCP / PCP 切分》 讲过 spec 和块池）。`customize_spec` 把 spec 改成每 token 1 个「头槽」、每头槽 `page_bytes(H) / 128` 字节，于是 vLLM 给每个块每层分配一页 `page_bytes(H)` 字节。例子 H = 2：页是 95,232 B，raw 是 2 × 2 × 128 × 128 × 2 = 131,072 B，同样的字节数多装 1.376× 个块。它顺带调 `registry.record_geometry(H, fp8)`，给 7.1 节的 connector 用。
3. **布局**：`supported_kv_cache_layouts` 只返回 `LBNHC`（层在外），每层的块在一个连续张量里，第 b 块的页就是这个张量的第 b 行。
4. **限制**（`FragPackMetadataBuilder.__init__` 和 backend 的类方法检查）：V2 model runner；不支持投机解码；`--block-size 128`；head_dim 128；GQA 组大小整除 8；SM90（TMA 和 FA3 prefill）；TP 整除 KV 头数；`LMCACHE_FRAGPACK_TABLES` 指向标定好的 `tables.npz`，表的 dtype 要和 cache 一致。每个 TP rank 只取自己那几个头的表：vLLM 给 rank r 的是第 [r × H, (r + 1) × H) 个 KV 头。

池能放多少：harness 给 CUSTOM 的 arm 少 858 MiB KV 预算（`FRAGPACK_RESERVED_BYTES`），注释的理由是 backend 在 vLLM 定完池大小之后才分配 staging、溢出池和 decode workspace，扣掉它让各 arm 用的显存一样。按本文的默认配置算，staging 768 MiB + 溢出池 256 MiB + prefill workspace 256 MiB 已超过 858 MiB（6.3、6.5 节），这个常数应是在更小的 `--max-num-seqs` 下定的。[推断] (54 GiB − 858 MiB) / (48 层 × 95,232 B) = 12,487 个块 = 1.60M token；raw 是 54 GiB / (48 × 131,072) = 9,216 个块 = 1.18M token，实际多 1.355×。 **读码** `benchmarks/lossless_kv/arms.py`

6.2 record 和 page：按平均长度定页大小，CAS 抢空间

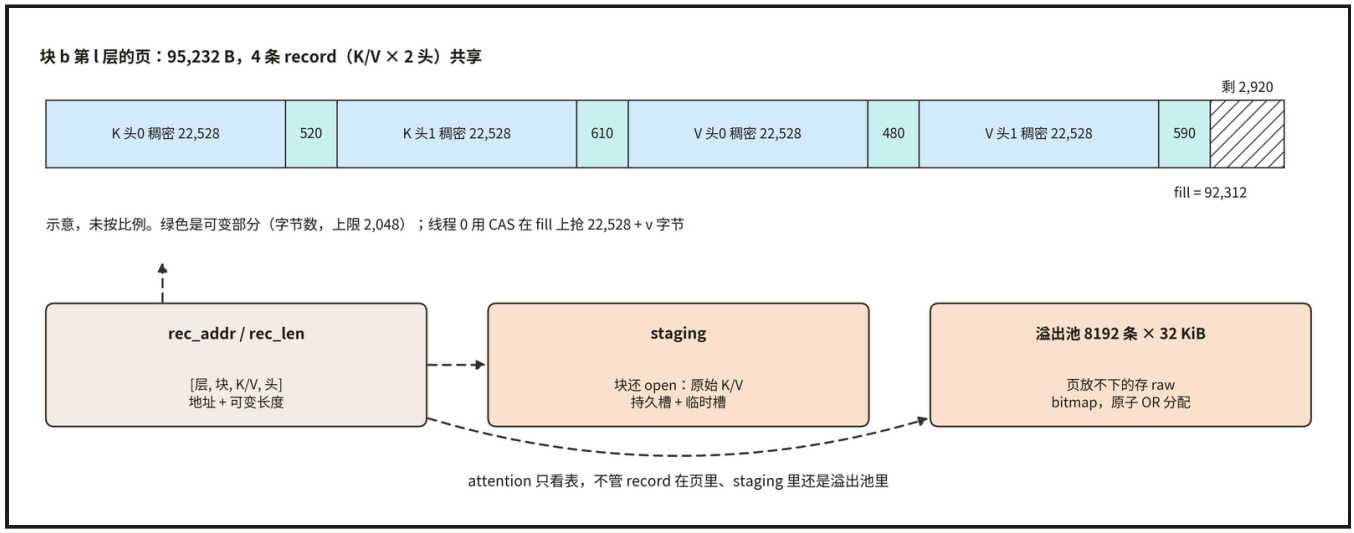


图 6-2 一个块一层的页里放 4 条 record

一条 record：一个块、一层、K 或 V、一个 KV 头，128 token × 128 维，原始 32 KiB。它和 tier record 用同一个 3 bit 主码，但标量按 MMA fragment 的顺序排，每个线程用 16 字节 load 读自己的 64 个标量（论文第 IV 节，《ExactKV 论文稿：无损 KV 压缩进缓存层和 attention》 5.2 节讲过）。

部分	BF16	FP8
稠密部分（固定）	22,528 B：低字节 16 KiB + 主码 6 KiB	12,288 B：尾数 6 KiB + 主码 6 KiB
可变部分上限 `var_cap`	2,048 B：48 B 头、每个转义一个 nibble、raw 标量	4,080 B
raw record	32,768 B	16,384 B

码表是每（层，K/V，头）一张 22 项：7 个主码 + 15 个扩展值，FP8 还有每通道一个指数偏移（2.5 节）。它和 tier 的码本来自同一次标定，`LMCACHE_FRAGPACK_TABLES` 指向 artifact 目录里的 `tables.npz`（2.2 节）。转义用一个 4 bit nibble 编，16 个值里 0–14 选扩展值、15 表示 raw 标量，所以扩展值正好 15 个，比 tier 的次码（3 bit，7 个）多一倍，raw 更少。

页大小：`page_bytes(H) = 2H × (稠密部分 + VAR_SHARE)`。BF16 的 `VAR_SHARE = 1280`：一页 4 条 record 共享 4 × 1280 = 5,120 B 的可变空间，平均每条 1280，但单条可以用到 2,048。论文在 agent KV 上测过：1280 B 份额不溢出，1024 B 约 2% 的 record 溢出，而溢出的 record 每条占 32 KiB。[1]FP8 的 `FP8_VAR_SHARE = 2560`，页是 59,392 B，对 raw 的 65,536 只多装 1.10×：FP8 的 4 位指数分布更散，转义多。

写进页：encode kernel 一条 record 一个 CTA（256 线程），块内扫描算出可变部分长度 v，再由线程 0 对这一页的填充计数器 `fill[层, 块]` 做 CAS（`page_claim`）抢 22,528 + v 字节。

- 抢到：record 写在页里 `fill` 的位置。
- 编不了（可变部分超过 2,048，或某个 warp 有超过 32 个 raw 标量）：存 raw，32 KiB，先试页里放不放得下。
- 页里放不下（不管压缩还是 raw）：存 raw 进溢出池。溢出池默认 8192 条（`LMCACHE_FRAGPACK_OVERFLOW`，BF16 是 8192 × 32 KiB = 256 MiB），用 bitmap 管理，原子 OR 分配；7.3 节的 import 也从这里分配，所以两条 stream 可以并发分配。
- 溢出池也满：计 `kErrOverflow`，这条 record 丢了。`HbmKvStore.poll` 每 16 步异步读一次计数器（每次读上一次发起的快照，所以从不阻塞主机），发现丢数据就抛 `StoreError`，不让服务带着坏 KV 继续跑。8192 条和每 16 步都没有写理由：1280 B 份额下溢出很少，8192 条（256 MiB）是余量；16 步让轮询的开销可以忽略。[推断]

CAS 而不是原子加：原子加失败后计数器已经加上去了，同一页后面更小的 record 就放不进来；CAS 失败计数器不变。

定位 record：每层两张表 `rec_addr[块, 2, H]`（record 的绝对设备地址）和 `rec_len`（稠密部分之后的字节数，raw 用一个大于 `var_cap` 的特殊值）。record 可以在页里、在 staging（块还没写满）、在溢出池，attention 只看表，不关心它在哪。没写过的块（vLLM 的 null 块、dummy run）指向一条全零的 raw record，所以越不了界。

6.3 每一步：prepare、逐层 write / attend / encode、commit

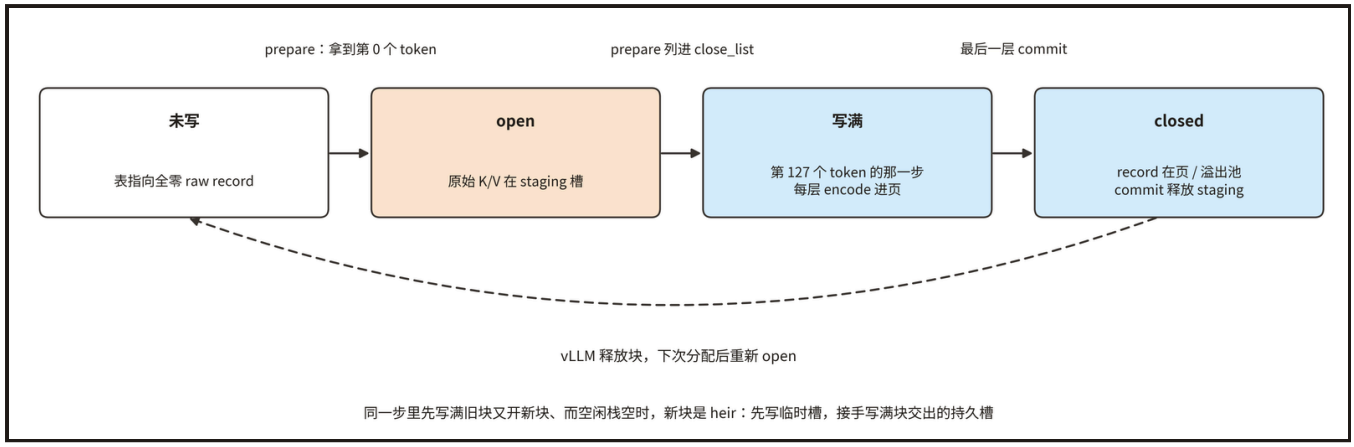


图 6-3 一个块在 pool mode 里的状态

只有写满 128 个 token 的块才编码；还在写的块（open）存原始 K/V，放在 staging 里。

`HbmKvStore`（hbm_kv.py，每个 worker 一个）管这一切：每层的 `rec_addr` / `rec_len` 表、页的填充计数、staging 槽、溢出池，以及下面的 prepare、write、encode、commit。`FragPackMetadataBuilder` 是 vLLM 每步建 attention metadata 时调用的对象，它持有 store；`FragPackAttentionImpl.forward` 是每层的 attention。

```
FragPackMetadataBuilder.build(common_attn_metadata)          每步一次，在 forward 之前
├── _store()                      第一次调用时建 HbmKvStore（KV cache 绑定后），registry.regi
├── 数这一步跨过 128 边界的块数 → max_close
├── store.prepare(slot_mapping, num_tokens, _TRACKER.live_blocks())   1 次 H2D + 1
│   ├── GC：不在 live_blocks 里的块释放 staging 槽
│   ├── 拿到第 0 个 token 的块：从空闲栈领一个 staging 槽，表指向 staging
│   └── 拿到第 127 个 token 的块：列进 close_list
└── 每 16 步 store.poll()
FragPackAttentionImpl.forward(layer, q, k, v, …)                每层一次
├── store.write(layer, k, v, slot_mapping)   新 token 写进 staging（FP8 按 x / scale
├── decode 行：decode_attention(...)          6.4 节
├── prefill 行：store.gather → flash_attn_varlen_func    6.5 节
├── store.encode(layer, max_close)            这一层里本步写满的块编码进页
└── 最后一层：store.commit()                  释放本步写满的块的 staging
```

staging 有两种：

- 持久槽 `[S_p, 层, 2, H, 32 KiB]`：跨步保持 open 的块（decode 的尾块、chunked prefill 的边界块）。`S_p = max_num_seqs`，因为一个运行中的请求在两步之间最多只有一个没写满的块。例子 `--max-num-seqs 128`：128 × 48 × 2 × 2 × 32 KiB = 768 MiB。
- 临时槽 `[S_t, 2, H, 32 KiB]`：同一步里打开又写满的块，只要一层的量，逐层复用。`S_t = max_tokens / 128 + 2 + S_p`。

heir：一个请求的 chunk 在这一步里先写满了旧尾块、又开了新尾块，就要两个槽。空闲栈空了时，新尾块成为 heir（继承者），先写进临时槽；本步写满的某个块在每层 encode 时把它的持久槽交给 heir（拷过去），commit 时 heir 正式接手。所以 `S_p` 只要覆盖步与步之间的 open 块，不用覆盖步内的峰值。heir 多于可交出的槽时计 `kErrStaging`，`poll` 会抛错；`S_p ≥ max_num_seqs` 时不会发生。

CUDA graph：每个 kernel 的 launch 形状只取决于补齐后的 batch 形状（`max_close` 在 decode-only 的批里就是 decode 行数），per-layer 的操作都可以录进 graph。`prepare` 在 graph 外、`build` 里急切执行，所以 FULL graph 重放之前它已经跑过了。`_cudagraph_support = UNIFORM_SINGLE_TOKEN_DECODE`：只有纯 decode 步走 graph。

6.4 OpenBlocks：从调度器输出镜像每个请求的块表

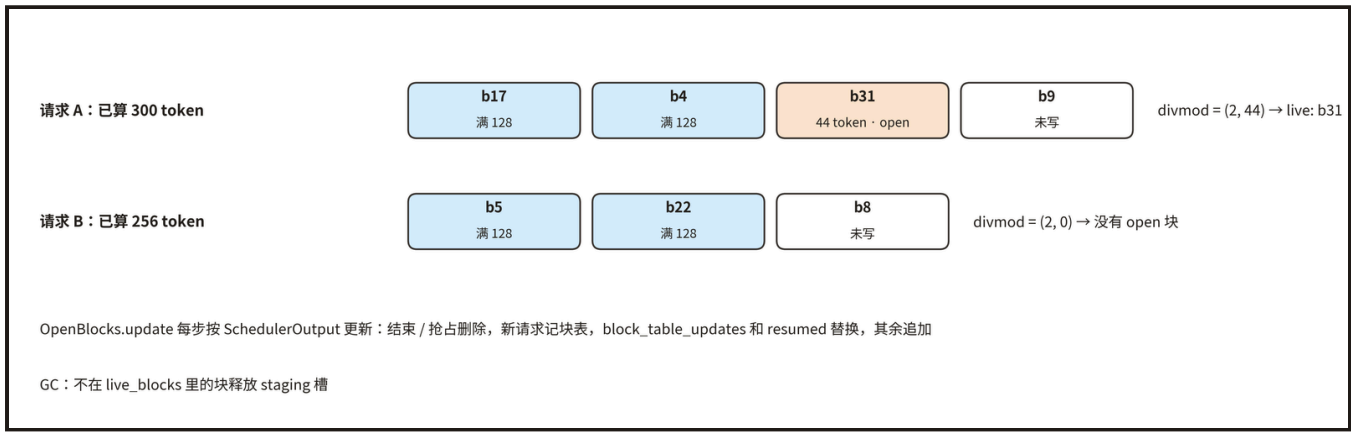


图 6-4 OpenBlocks 怎么找出每个请求唯一的 open 块

问题：GC 要知道哪些块还会被写，可 vLLM 不告诉 attention backend 块什么时候被释放；一个块被释放、又分给别的请求时，它的旧 staging 槽必须先收回。

做法：`_install_tracker` 包一层 `GPUModelRunner.execute_model`，每步（非 dummy run）先把 `SchedulerOutput` 喂给 `OpenBlocks.update`：

- 结束和被抢占的请求：删掉。
- 新请求：记下块表和已算 token 数。
- 有 `block_table_updates` 的请求：整张表替换；resumed 的请求：用新块表替换；其余的：追加新块。

`live_blocks()` 对每个请求算 `index, offset = divmod(已算 token 数, 128)`，只有 `offset ≠ 0` 时第 `index` 个块是 open 的。例子：一个请求已算 300 个 token，`divmod(300, 128) = (2, 44)`，第 2 个块写了 44 个 token，是唯一保留 staging 的块。

为什么不能宽松一点：如果把「请求还持有的块」都当成 live，一个在 open 时被释放、又被分给别的请求、而新主人还没写到它的块，会一直占着旧槽，步与步之间的 open 块就会多于请求数，staging 被用光。论文记的现象是用更宽的判断时，32 个请求就把 staging 用完了。[1]

6.5 decode attention：所有请求共用一个段长

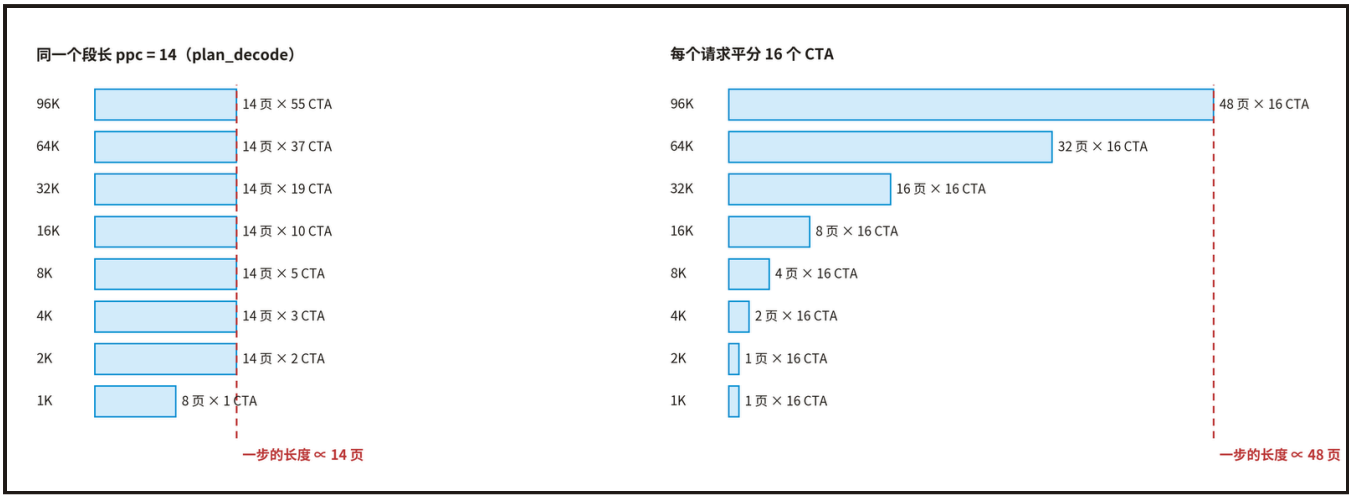


图 6-5 8 个上下文长短不一的请求怎么分 CTA

一个 CTA 做什么：256 线程，负责一个（请求，KV 头）的一段连续页，处理 GQA 组里全部 8 个 query 头。线程 0 用 TMA 把下一页的 K、V record 预取进共享内存，每个线程把自己的 64 个标量解码成 `mma.m16n8k16` 的 A fragment，算 $S^T = K\,Q^T$（8 个 query 头正好填满 n = 8），在线 softmax，再解 V 算 $O^T \mathrel{+}= V^T\,P^T$。最后一个 `combine` kernel 合并每个请求各段的部分结果。解出的值只在寄存器里，不回写显存。

读码 `fused_attn_decode.py` `fused_attn_decode.cu`

CTA 数：`decode_units(batch, H, SM 数) = max(batch, 2 × SM / H)`。每 SM 放 2 个 CTA（`__launch_bounds__(256, 2)`），全部 KV 头的 CTA 正好一波。例子 132 个 SM、H = 2：每个 KV 头 132 个单元，一层 264 个 CTA。只依赖 batch 大小，所以 CUDA graph 里形状固定。

段长怎么定：`plan_decode`（一个 256 线程的 CTA）在设备上二分出最小的段长 ppc（每段多少页），使所有请求的段数之和不超过单元数；每个请求的段数 = ⌈页数 / ppc⌉，至少 1。第 0 层算一次（`replan = li == 0`），其余层复用。

例子：8 个 decode 请求，上下文 96K、64K、32K、16K、8K、4K、2K、1K token，即 768、512、256、128、64、32、16、8 页，共 1,784 页。下界 ⌈1784 / 132⌉ = 14；ppc = 14 时段数 55 + 37 + 19 + 10 + 5 + 3 + 2 + 1 = 132，正好放下，所以 ppc = 14，最长的请求分到 55 个 CTA。如果每个请求平分 132 / 8 = 16 个 CTA，最长的那个每个 CTA 要读 768 / 16 = 48 页，一步的长度由它决定，是 14 页的 3.4 倍。论文测的是这种平分比 FA3 慢 1.4–1.8×。[1]

6.6 prefill：解进 BF16 workspace 再跑 FA3

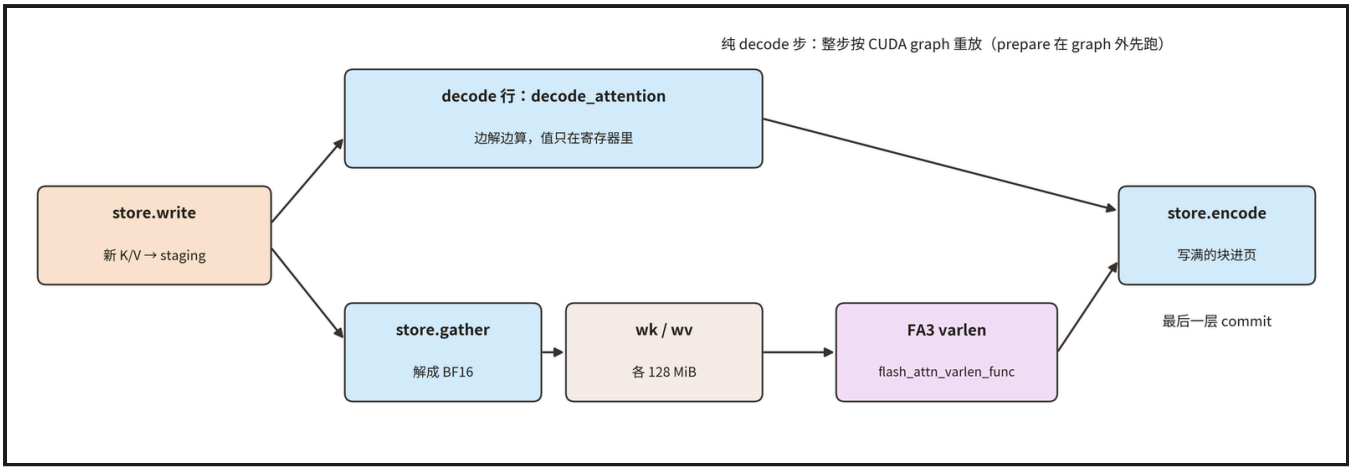


图 6-6 prefill 行和 decode 行在一层里怎么走

prefill 是计算密集的，不融合：`store.gather` 把这一层要读的上下文页解成 token 主序的 BF16，写进两块 workspace `wk`、`wv`（各 `[[max_model_len / 128], 128, H, 128]`），再调 vLLM 自带的 `flash_attn_varlen_func`（FA3）。例子 `--max-model-len 262144`：2048 页 × 128 × 2 × 128 × 2 B = 128 MiB 一块，两块 256 MiB。FP8 record 解出的是不带 scale 的 E4M3 值，K 的 scale 并进 softmax scale，V 的 scale 乘在输出上。

论文估算融合这一步最多省 prefill 的 3%，所以没做。[1]

7. pool mode 接 LMCache：块当 blob 存

pool mode 下 GPU 池里的块已经是 FragPack record，而且一个块的 record 不在固定位置（页里、溢出池里都可能），LMCache 不能按原始布局拷。所以 cache server 不跑 codec，用一个 export kernel 把一个写满的块全部层的 record 拷成一个连续的 blob 存进 L1/L2，retrieve 时用 import kernel 装回目标块。两个 kernel 都在 cache server 进程里跑，读写的是 vLLM worker 的显存。

读码 `lmcache/v1/multiprocess/fragpack_blob_transfer.py`
`lmcache/v1/lossless_kv/fragpack/hbm_kv.py`

7.1 握手：layout hint 先到，store 的表后到

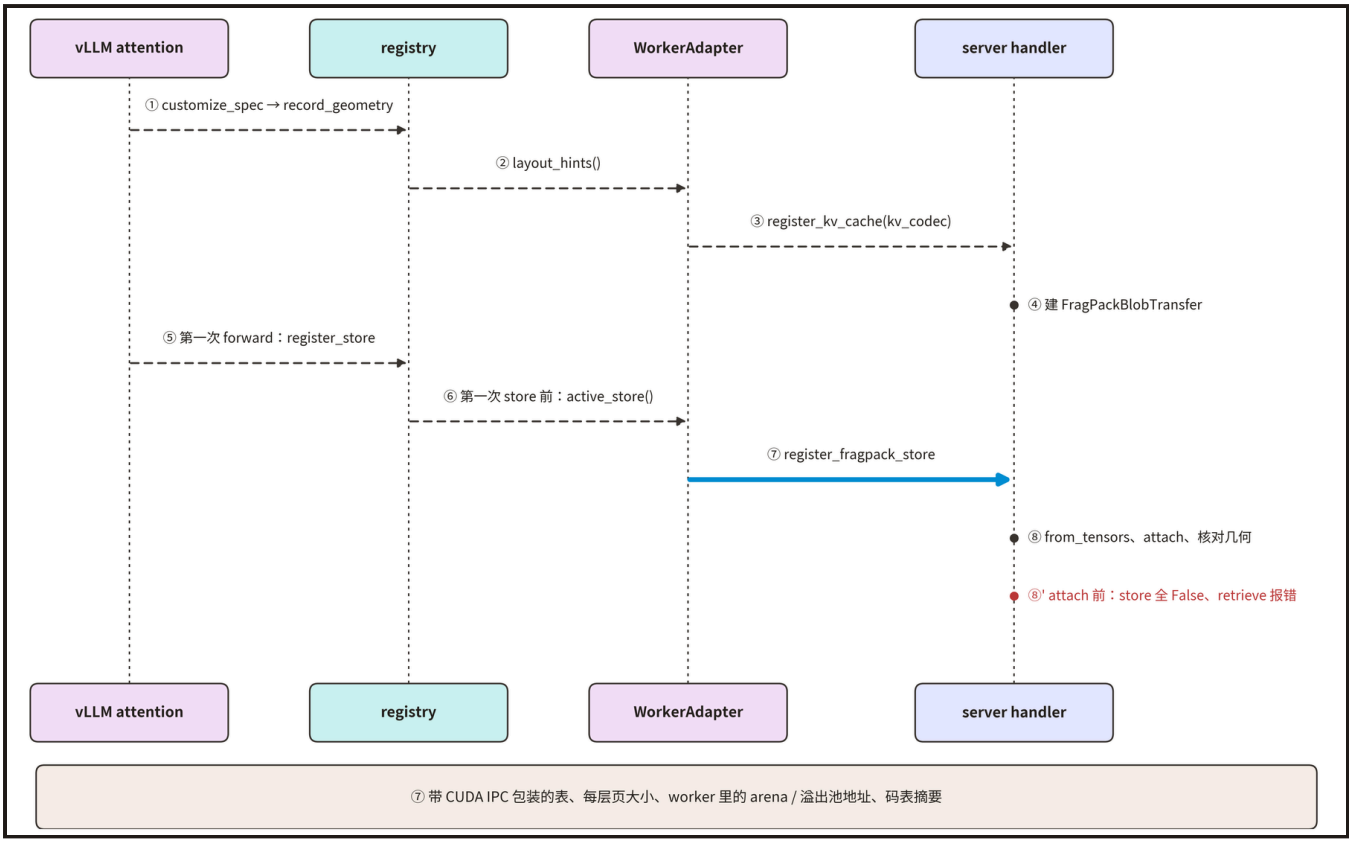


图 7-1 worker 和 cache server 之间的两次注册

问题：vLLM 注册 KV cache 时 FragPack 的 store（`HbmKvStore`）还不存在，它在第一次 forward 的 metadata build 里才建（6.3 节）；而 cache server 在注册时就要决定用哪种 transfer，到第一次 store 时还要拿到 store 的表。

做法：分两步，中间靠 `fragpack/registry.py` 交接。这个模块不 import CUDA 扩展，connector 不用 FragPack 时也能安全 import 它。

1. vLLM 定 KV spec 时 `customize_spec` 调 `record_geometry(H, fp8)`。
2. worker 注册 KV cache：`vllm_layout_hints` 先合并 `registry.layout_hints()`，得到 `{"kv_codec": "fragpack", "num_kv_heads": 2}`，随 `register_kv_cache` 发给 server。server 的 `_create_record_transfer` 看到 `kv_codec` 就建 `FragPackBlobTransfer`（1.3 节）。它要求 LMCache 的 chunk 正好是一个 128 token 的块，否则注册失败。
3. 第一次 forward：`FragPackMetadataBuilder._store()` 建 store，`registry.register_store(store)`。
4. worker 每次 `submit_store_request` / `submit_retrieve_request` 前调 `_share_fragpack_store()`：hint 里有 `kv_codec`、`registry.active_store()` 不是 None、而且和上次发过的不是同一个 store，就发 `register_fragpack_store` RPC（新加的 RPC，gRPC proto 里也加了），带 store 的表、每层页大小、溢出池和每层 arena 在 worker 里的地址、码表摘要。CUDA IPC 包装的张量（`wrap_one_kv_cache`）在 server 里映射成它自己的张量。等 `mq_timeout`，超时 `ConnectionError`。

5. server 的 handler 用 `BlobTables.from_tensors` 重建表，`FragPackBlobTransfer.attach` 核对层数、头数、FP8 和 blob 大小，不一致 `ValueError`。

attach 之前：store 不入队任何块（返回全 False），retrieve 抛 `RuntimeError`（vLLM 重算）。store 重建（KV cache 重新绑定）或 worker 重新注册后，`_fragpack_store_sent` 清空，下次再发。

两套地址：export 和 import 由 `BlobPort`（hbm_kv.py，在一个进程里对一个 store 的表做 export / import 的对象）发起。`rec_addr` 里是 worker 进程里的设备地址，server 进程里同一块显存映射到另一个地址。它拿着两套：`arena_rec` / `ovf_rec`（worker 的地址）和 `arena_dev`（server 映射出的地址），kernel 用「地址 − worker 的 arena 起点 + server 的 arena 起点」换算。表本身（`rec_addr`、`rec_len`、`fill`、溢出 bitmap）也是共享的，所以 server 的 import 改的就是 vLLM attention 读的那张表。

7.2 blob 格式：4 KiB 头，后面接 192 条 record

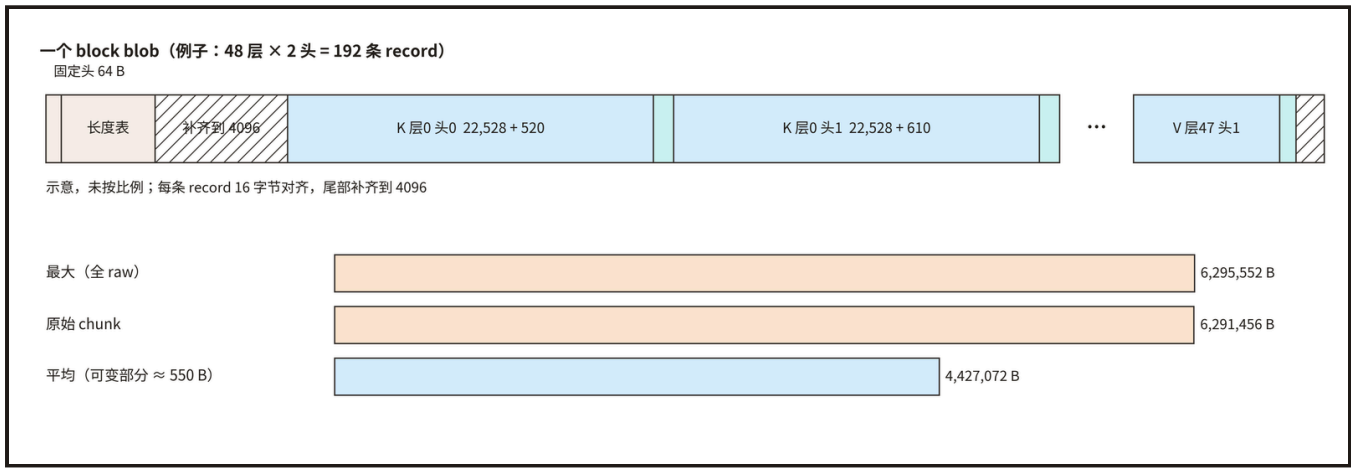


图 7-2 例子里一个 block blob 的字节布局

段	大小（例子）	内容
固定头	64 B	magic `FPKB`、版本 2、层数 L、头数 H、blob 字节数、码表摘要、record 格式（1 = FP8）
长度表	8 × L × H = 768 B	int32 `rec_len[L, 2, H]`
补齐	到 4096 B	`blob_head_bytes`
record	2 × L × H = 192 条	按（层，K/V，头）顺序，每条 22,528 + `rec_len` 字节（raw 整条 32 KiB），都 16 字节对齐
尾部补齐	到 4096 B	长度是 4 KiB 的倍数，O_DIRECT 可用

一个 blob 多大：最大是全部 raw，4,096 + 192 × 32,768 = 6,295,552 B（`blob_slot_bytes`），比原始 chunk 多 4 KiB 的头，这也是 store pool 和 retrieve staging 的槽大小。按论文 agent KV 上可变部分平均约 550 B，一个 blob 约 4,096 + 192 × 23,078 ≈ 4.43 MB，是原始的约 0.70。

码表摘要：store 建时用 blake2b 对码表和 FP8 偏移算一个 8 字节摘要。import 校验它，码表不同的 blob（比如换了 `LMCACHE_FRAGPACK_TABLES` 后从 L2 读回旧 blob）会被拒绝，而不是解出错的 KV。

7.3 store 和 retrieve：export 进 pool，import 失败就重算

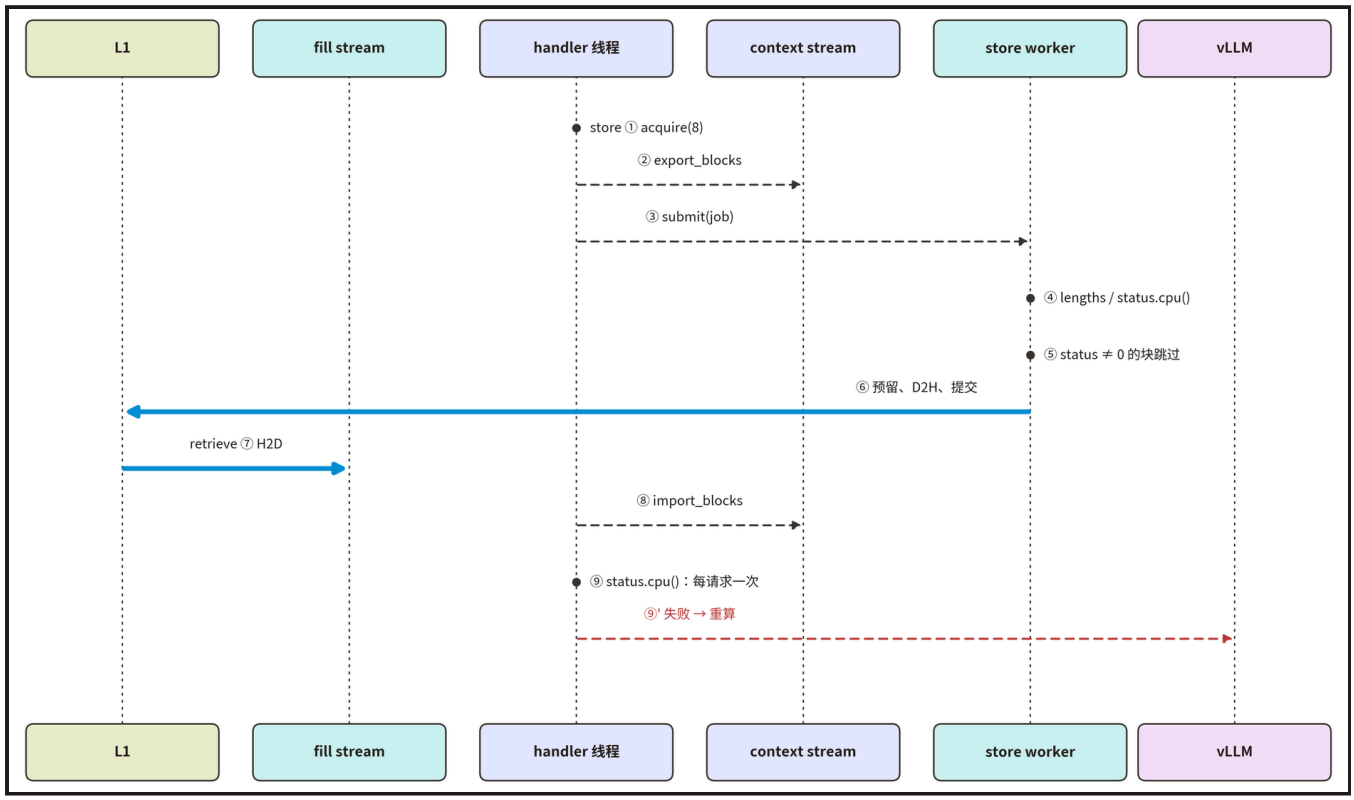


图 7-3 blob 的 store 和 retrieve

```
store（handler 线程）
└── FragPackBlobTransfer.submit_store(块号, keys, skip_mask, tally)
    ├── 没 attach → 全 False
    ├── l1_contains 跳过已在 L1 的
    └── 每 8 个一批
        ├── worker.acquire(8)                    RecordStoreWorker，和 4.3 节同一个类，
        ├── port.export_blocks(块, pool, 偏移)   grid (8, 48)：一个 CTA 拷一个块一层的 4
        │                                          输出设备上的 lengths（取整到 4 KiB）和
        ├── ready = Event().record(context stream)
        └── worker.submit(job)，records 回调 = _blob_records(lengths, status)
store worker 线程
└── _store：等 ready → lengths.cpu()、status.cpu() → status ≠ 0 的块跳过并记 warning →
retrieve（handler 线程）
└── FragPackBlobTransfer.retrieve(块号, objs, skip_first_n_tokens)
    ├── skip 不是 128 的倍数 → ValueError
    └── 每 8 个一批，四套 staging 轮转
        ├── fill stream 等这一套上次的 import 用完 → H2D 拷 blob
        ├── context stream 等 fill event → port.import_blocks(staging, 偏移, 块)
        └── 记这一套的 done event
    └── 全部 status 拼起来 .cpu()     整个请求同步一次；有非零 → RuntimeError，vLLM 重算
```

status 位：1 头坏（magic、几何、摘要不对）、2 长度坏、4 溢出池满、8 块没关（导出时块还 open，或从没写过）。

export 失败不算错：vLLM 只存写满的块，status 8 说明这个块在 store 跑之前就被释放或重新打开了。跳过它只是少存一块，之后最多重算。

import 为什么要同步：import 可能真的失败（溢出池满、码表不对），失败的块的表没有写，attention 读它会读到旧数据。所以 retrieve 必须在返回之前知道结果，整个请求读一次 status。codec 的 retrieve 不需要这一步，因为它的解码不会失败（3.2 节的健壮性保证只写自己的块）。

import 的顺序：先把 record 按顺序装进目标块的页（页装不下的 raw record 去溢出池），最后才换表；目标块原来在溢出池里的 record 在换表之后才释放。所以一层失败时那一层的表不变，仍然拥有它的溢出槽。

四套 staging：最早 retrieve 只有一块 staging，第 k + 1 批的拷贝要等第 k 批 import 完。改成四套（`RETRIEVE_STAGING_SETS = 4`，每套 8 × 6,295,552 B ≈ 48 MiB）和 fill stream 之后，AgentX 实验里 pool mode 的中位 TTFT 从 1.14 s 降到 0.97 s。[7] ［实测］ `RETRIEVE_STAGING_SETS = 4` 和 5.2 节的 `decode_sets = 4` 取同一个值，理由同那里。［推断］

和 tier 共用的部分：store worker（`RecordStoreWorker`）、`StoreTally`、按长度预留 L1、L2 的大小记录都是同一套代码，blob 和 tier record 在 L1/L2 看来都是「一维 uint8、大小不定的对象」。